70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A4

Edge AI va Cloud AI platformalarida sun'iy intellekt tizimlarini joylashtirish

Bajarish bo‘yicha qo‘llanma: 6 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A4
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A4
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu virtual laboratoriyada siz inferens modelini xizmat sifatida joylashtirish zanjirini jarayon ichida bajarasiz: FastAPI xizmati (/healthz, /readyz, /predict, /metrics) `TestClient` bilan sinaladi va yuklama ostida o'lchanadi, Dockerfile va Kubernetes manifestlari (Deployment/Service/HPA/ConfigMap) `pyyaml` bilan tahlil qilinib muhandislik qoidalariga tekshiriladi, HPA avtomasshtablash navbat modeli bilan simulyatsiya qilinadi, Cloud/Edge/Gibrid arxitekturalar TCO va kechikish kalkulyatori bilan solishtiriladi. Real labdan farqi: Docker/minikube/Jetson/Coral yo'q — konteyner va klaster o'rniga jarayon ichidagi xizmat va simulyatsiya; model — A3 dagi digits MLP (ONNX).

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A4`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A4 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. FastAPI inferens xizmati: probe'lar, predict, metrikalar
2. 2-qadam. Yuklama sinovi: ketma-ket va parallel so'rovlar, Little qonuni, paketlash
3. 3-qadam. Dockerfile tahlili va tasvir hajmini baholash
4. 4-qadam. Kubernetes manifestlari: pyyaml bilan validatsiya va 18 qoida
5. 5-qadam. HPA avtomasshtablash simulyatsiyasi (navbat modeli)
6. 6-qadam. Cloud vs Edge vs Gibrid: kechikish, trafik, energiya va TCO kalkulyatori
7. Qo‘shimcha mashqlar (10)
8. Mustaqil topshiriqlar — 20 variant

1 1-qadam. FastAPI inferens xizmati: probe'lar, predict, metrikalar

MAQSAD

Kichik MLP ONNX'ga eksport qilinadi va FastAPI xizmatiga o'raladi: `lifespan` ichida model yuklanadi va isitiladi, `/healthz` (liveness) va `/readyz` (readiness) ajratilgan, `/predict` JSON so'rovni qabul qilib top-3 va vaqt taqsimotini (preprocess/inference) qaytaradi, `/metrics` Prometheus matn formatida so'rovlar, xatolar, kechikish kvantillarini beradi. TestClient model yuklanmasdan oldin (`/readyz` → 503) va keyin (`with` blok) chaqiriladi.

> **Kutilgan natija:** Yuklanmasdan oldin /readyz 503, /healthz 200; keyin 200; 5 namunadan 4–5 tasi to'g'ri, 200 namunada ~90 % aniqlik; noto'g'ri so'rovlar 422; /metrics matni.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. FastAPI inferens xizmati: /healthz, /readyz, /predict, /metrics — jarayon ichida
TestClient bilan
import warnings; warnings.filterwarnings("ignore")
import os, time, json, numpy as np, torch, torch.nn as nn, onnxruntime as ort
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from fastapi.responses import PlainTextResponse
from fastapi.testclient import TestClient
from pydantic import BaseModel
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# --- A3 dan "optimallashtirilgan model": kichik MLP -> ONNX (digits, 8x8). Real labda bu
resnet50_int8.onnx
X, y = load_digits(return_X_y=True); X = (X / 16.0).astype(np.float32)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
model = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 10))
opt = torch.optim.Adam(model.parameters(), 3e-3); lossf = nn.CrossEntropyLoss()
Ttr, Tytr = torch.tensor(Xtr), torch.tensor(ytr)
for ep in range(4):
    perm = torch.randperm(len(Ttr))
    for i in range(0, len(Ttr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lossf(model(Ttr[idx]), Tytr[idx]).backward();
opt.step()
model.eval()
torch.onnx.export(model, Ttr[:1], "digits.onnx", input_names=["input"], output_names=
["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)

# --- 12 omilli ilova: barcha sozlamalar muhit o'zgaruvchilaridan (Docker/K8s'da env orqali
beriladi)
MODEL_PATH = os.getenv("MODEL_PATH", "digits.onnx")
ORT_THREADS = int(os.getenv("ORT_THREADS", "2"))
state = {"ready": False, "session": None, "sorovlar": 0, "xatolar": 0, "kechikish": []}

@asynccontextmanager
async def lifespan(app):
    """startup: model yuklash + isinish; shutdown: tozalash. K8s readiness probe shu holatga
```

```
qaraydi."""
    t0 = time.perf_counter()
    so = ort.SessionOptions(); so.intra_op_num_threads = ORT_THREADS
    so.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
    state["session"] = ort.InferenceSession(MODEL_PATH, so, providers=["CPUExecutionProvider"])
    state["session"].run(None, {"input": np.zeros((1, 64), np.float32)})      # isinish —
birinchi so'rov sekin bo'lmasin
    state["ready"] = True
    print(f"[startup] model yuklandi: {MODEL_PATH}, {ORT_THREADS} oqim, {(time.perf_counter() -
t0) * 1e3:.0f} ms")
    yield
    state["ready"] = False
    print("[shutdown] xizmat to'xtadi")

app = FastAPI(title="digits-infer", lifespan=lifespan)

class Sorov(BaseModel):
    pixels: list[float]          # 64 ta qiymat, 0..16

@app.get("/healthz")             # liveness: jarayon tirik
def healthz():
    return {"ok": True}

@app.get("/readyz")              # readiness: model yuklangan va so'rov qabul qila oladi
def readyz():
    if not state["ready"]:
        raise HTTPException(503, "model not loaded")
    return {"ready": True, "model": os.path.basename(MODEL_PATH), "threads": ORT_THREADS}

@app.post("/predict")
def predict(s: Sorov):
    if not state["ready"]:
        raise HTTPException(503, "not ready")
    if len(s.pixels) != 64:
        state["xatolar"] += 1
        raise HTTPException(422, "64 ta piksel kerak")
    t0 = time.perf_counter()
    x = (np.asarray(s.pixels, np.float32) / 16.0)[None]                          # oldindan
ishlov
    t1 = time.perf_counter()
    logits = state["session"].run(None, {"input": x})[0][0]                      # inferens
    t2 = time.perf_counter()
    top = logits.argsort()[-3:][::-1]
    state["sorovlar"] += 1; state["kechikish"].append((t2 - t0) * 1e3)
    return {"top3": [{"label": int(i), "score": round(float(logits[i]), 3)} for i in top],
            "timing_ms": {"preprocess": round((t1 - t0) * 1e3, 3), "inference": round((t2 - t1)
* 1e3, 3)}}

@app.get("/metrics", response_class=PlainTextResponse)   # Prometheus matn formati (4 signal:
so'rovlar, xatolar, kechikish, resurs)
def metrics():
    k = state["kechikish"] or [0.0]
    return (f"# TYPE infer_requests_total counter\ninfer_requests_total {state['sorovlar']}\n"
            f"# TYPE infer_errors_total counter\ninfer_errors_total {state['xatolar']}\n"
            f"# TYPE infer_latency_ms summary\ninfer_latency_ms{{quantile=\"0.5\"}}
{np.percentile(k, 50):.3f}\n"
            f"infer_latency_ms{{quantile=\"0.95\"}} {np.percentile(k, 95):.3f}\n"
            f"# TYPE infer_ready gauge\ninfer_ready {int(state['ready'])}\n")

print("=== FastAPI xizmati ===")
# Lifespan ishga tushmagan mijoz — Pod endi yaratilgan, model hali yuklanmagan holat
sovuq = TestClient(app)
r = sovuq.get("/readyz"); print(f"Model yuklanmasdan oldin GET /readyz -> {r.status_code}
```

```
{r.json()}  (K8s trafik yubormaydi)")
print(f"                        GET /healthz -> {sovuq.get('/healthz').status_code}  (Pod
tirik — o'ldirilmaydi)")

with TestClient(app) as c:            # with-blok lifespan'ni ishga tushiradi (startup -> yield
-> shutdown)
    print("GET /readyz ->", c.get("/readyz").status_code, c.get("/readyz").json())
    print("\nPOST /predict (5 ta test namunasi):")
    togri = 0
    for i in range(5):
        r = c.post("/predict", json={"pixels": (Xte[i] * 16).tolist()})
        j = r.json(); bashorat = j["top3"][0]["label"]; togri += bashorat == yte[i]
        print(f"  #{i} haqiqiy {yte[i]}  bashorat {bashorat}  {'OK' if bashorat == yte[i] else
'XATO'}  timing {j['timing_ms']}")
    print("Noto'g'ri so'rov (3 ta piksel) ->", c.post("/predict", json={"pixels": [1, 2,
3]}).status_code)
    print("Yaroqsiz JSON             ->", c.post("/predict", json={"foo": 1}).status_code)
    # Butun test to'plami bo'yicha aniqlik — xizmat orqali
    acc = np.mean([c.post("/predict", json={"pixels": (Xte[i] * 16).tolist()}).json()["top3"]
[0]["label"] == yte[i] for i in range(200)])
    print(f"\nXizmat orqali 200 namunada aniqlik: {acc * 100:.1f} %")
    print("\nGET /metrics:"); print(c.get("/metrics").text)
    print("OpenAPI yo'llari:", sorted(p for p in c.get("/openapi.json").json()["paths"]))
print("\nIzoh: real labda bu xizmat uvicorn bilan 8000-portda, Docker ichida ishlaydi; bu yerda
TestClient uni jarayon ichida chaqiradi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== FastAPI xizmati ===`, `GET /readyz -> 200`, `infer_requests_total`, `OpenAPI yo'llari`

TAJRIBA QILIB KO‘RING

- `/predict` ga `batch` maydonli variant qo'shing (bir so'rovda N ta rasm).
- `ORT_THREADS` ni `os.environ` orqali 1 ga qo'ying — timing qanday o'zgaradi?
- `/metrics` ga xotira (`psutil.Process().memory_info().rss`) gauge'ini qo'shing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Yuklama sinovi: ketma-ket va parallel so'rovlar, Little qonuni, paketlash

MAQSAD

`hey -c N` o'rniga `ThreadPoolExecutor` bilan 1, 2, 4, 8, 16 parallel mijoz simulyatsiya qilinadi: throughput, p50/p95/p99 va Little qonuni (N = X·R) tekshiriladi; to'liq kechikish ichidagi sof inferens ulushi ko'rsatiladi; 64 ta alohida so'rov va bitta paketli so'rov solishtiriladi; p95 ≤ 50 ms SLO tekshiriladi. Grafik: throughput va kechikish.

> **Kutilgan natija:** c=1: p50 ~2–3 ms, ~400 so'rov/s (sof inferens 0,1 ms — qolgani HTTP/JSON); throughput 2 yadroda to'yinadi, p99 mijozlar bilan chiziqli o'sadi; paket 20–30x tezroq.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Yuklama sinovi: ketma-ket va parallel so'rovlar, p50/p95/p99, throughput, Little
qonuni
import warnings; warnings.filterwarnings("ignore")
import os, time, numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as ort
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from concurrent.futures import ThreadPoolExecutor
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel
from sklearn.datasets import load_digits

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# Model: latency uchun o'qitish shart emas — tasodifiy vaznli MLP -> ONNX
model = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 10)).eval()
torch.onnx.export(model, torch.zeros(1, 64), "digits.onnx", input_names=["input"],
output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)
X, _ = load_digits(return_X_y=True)

state = {"ready": False, "session": None}

@asynccontextmanager
async def lifespan(app):
    so = ort.SessionOptions(); so.intra_op_num_threads = 1       # har so'rov 1 oqim — parallel
so'rovlar bir-biriga xalaqit bermasin
    state["session"] = ort.InferenceSession("digits.onnx", so, providers=
["CPUExecutionProvider"])
    state["ready"] = True
    yield

app = FastAPI(lifespan=lifespan)

class Sorov(BaseModel):
    pixels: list[float]

@app.post("/predict")
def predict(s: Sorov):
    if not state["ready"]: raise HTTPException(503)
```

```
    t0 = time.perf_counter()
    x = (np.asarray(s.pixels, np.float32) / 16.0)[None]
    logits = state["session"].run(None, {"input": x})[0][0]
    return {"label": int(logits.argmax()), "inference_ms": (time.perf_counter() - t0) * 1e3}

@app.post("/predict_batch")            # paketli so'rov: bir HTTP chaqiruvda N ta rasm
def predict_batch(s: list[Sorov]):
    x = np.asarray([q.pixels for q in s], np.float32) / 16.0
    return {"labels": state["session"].run(None, {"input": x})[0].argmax(1).tolist()}

def foiz(t):
    t = np.array(t) * 1e3
    return dict(p50=np.percentile(t, 50), p95=np.percentile(t, 95), p99=np.percentile(t, 99))

print("=== Yuklama sinovi ===")
with TestClient(app) as c:
    body = {"pixels": X[0].tolist()}
    for _ in range(50): c.post("/predict", json=body)                  # isinish

    # 1) Ketma-ket 300 so'rov: to'liq kechikish va uning ichidagi sof inferens
    t, inf = [], []
    for _ in range(300):
        a = time.perf_counter(); r = c.post("/predict", json=body);
t.append(time.perf_counter() - a)
        inf.append(r.json()["inference_ms"])
    f = foiz(t)
    print(f"Ketma-ket (c=1): p50 {f['p50']:.2f} ms  p95 {f['p95']:.2f} ms  p99 {f['p99']:.2f}
ms  -> {1 / np.mean(t):.0f} so'rov/s")
    print(f"  shundan sof inferens p50 {np.percentile(inf, 50):.3f} ms; qolgani ({f['p50'] -
np.percentile(inf, 50):.2f} ms) — HTTP, JSON, pydantic validatsiya")

    # 2) Parallel mijozlar (hey -c N o'rniga ThreadPoolExecutor): throughput va kechikish
    rows = []
    for cc in (1, 2, 4, 8, 16):
        n = 60 * cc
        def bitta(_):
            a = time.perf_counter(); c.post("/predict", json=body); return time.perf_counter()
- a
        a0 = time.perf_counter()
        with ThreadPoolExecutor(cc) as ex: tt = list(ex.map(bitta, range(n)))
        wall = time.perf_counter() - a0; f = foiz(tt); thr = n / wall
        rows.append(dict(mijozlar=cc, sorov_s=round(thr), p50_ms=round(f["p50"], 2),
p95_ms=round(f["p95"], 2), p99_ms=round(f["p99"], 2),
                         little_N=round(thr * np.mean(tt), 1)))
    df = pd.DataFrame(rows); print("\nParallel mijozlar:"); print(df.to_string(index=False))
    print("little_N = throughput x o'rtacha kechikish = tizimdagi so'rovlar soni (Little
qonuni) — mijozlar soniga yaqin bo'lishi kerak.")
    print("Kuzatuv: 2 yadro va GIL tufayli throughput tez to'yinadi, mijozlar ko'paygani sari
faqat navbat (p99) o'sadi.")

    # 3) Paketlash: 64 ta rasm bitta so'rovda vs 64 ta alohida so'rov
    paket = [{"pixels": X[i].tolist()} for i in range(64)]
    a = time.perf_counter()
    for _ in range(10): c.post("/predict_batch", json=paket)
    tb = (time.perf_counter() - a) / 10
    a = time.perf_counter()
    for _ in range(10):
        for q in paket: c.post("/predict", json=q)
    ts = (time.perf_counter() - a) / 10
    print(f"\n64 rasm: alohida so'rovlar {ts * 1e3:.1f} ms, bitta paketli so'rov {tb * 1e3:.1f}
ms -> {ts / tb:.1f}x tezroq (lekin paket to'lguncha kutish kerak).")

    # 4) SLO tekshiruvi
```

```
    SLO = 50.0
    print(f"\nSLO p95 <= {SLO} ms: ", ", ".join(f"c={int(r.mijozlar)}: {'OK' if r.p95_ms <= SLO
else 'BUZILDI'}" for _, r in df.iterrows()))

fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
ax[0].plot(df.mijozlar, df.sorov_s, "o-"); ax[0].set_xlabel("parallel mijozlar");
ax[0].set_ylabel("so'rov/s"); ax[0].set_title("Throughput"); ax[0].grid(alpha=.3)
ax[1].plot(df.mijozlar, df.p50_ms, "o-", label="p50"); ax[1].plot(df.mijozlar, df.p95_ms, "s-",
label="p95"); ax[1].plot(df.mijozlar, df.p99_ms, "^-", label="p99")
ax[1].axhline(SLO, color="r", ls="--", label="SLO"); ax[1].set_xlabel("parallel mijozlar");
ax[1].set_ylabel("ms"); ax[1].set_title("Kechikish"); ax[1].legend(); ax[1].grid(alpha=.3)
plt.tight_layout()
print("\nReal labda: hey -z 30s -c 8 -m POST ... yoki locust; bu yerda TestClient + oqimlar.
Docker --cpus=2 cheklovi sandbox'ning 2 yadrosiga mos.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Yuklama sinovi ===`, `Parallel mijozlar`, `little_N`, `SLO p95`

TAJRIBA QILIB KO‘RING

- SLO ni 10 ms ga tushiring — nechta mijozda buziladi?
- `so.intra_op_num_threads = 2` qiling va parallel natijalarni solishtiring (oqimlar raqobati).
- Har so'rovga 5 ms sun'iy ishlov (`time.sleep`) qo'shib ResNet-50 xizmatini taqlid qiling va Little qonunini qayta tekshiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Dockerfile tahlili va tasvir hajmini baholash

MAQSAD

Docker bo'lmagani uchun Dockerfile matni dasturiy tahlil qilinadi: ko'p bosqichli qurish, qotirilgan versiya, yengil asos, root bo'lmagan USER, pip/apt kesh, HEALTHCHECK, EXPOSE, exec CMD, qatlamlar tartibi, ENV sozlamalar — 11 qoida. Yaxshi (slim + ONNX Runtime) va yomon (pytorch:latest, root) variantlar solishtiriladi; tasvir hajmi, tortish vaqti va 20 tugunli klasterga tarqatish hajmi baholanadi.

> **Kutilgan natija:** Yaxshi variant 11/11, yomon 1/11; hajm bahosi ~270 MB vs ~8 GB (30x); 100 Mbit/s da tortish 22 s vs 11 daqiqa.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Dockerfile tahlili: ko'p bosqichli qurish qoidalari va tasvir hajmini baholash
(Docker'siz)
import re, pandas as pd

# Ikki variant: yaxshi (ko'p bosqichli, root'siz, ORT) va yomon (bitta bosqich, root, to'liq
PyTorch)
YAXSHI = """
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
RUN useradd -m -u 10001 infer && mkdir -p /models && chown infer /models
COPY --from=builder /install /usr/local
WORKDIR /app
COPY app/ /app/
USER infer
ENV MODEL_PATH=/models/digits_int8.onnx ORT_THREADS=2 PYTHONUNBUFFERED=1
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --start-period=20s CMD python -c "import
urllib.request;urllib.request.urlopen('http://127.0.0.1:8000/readyz')"
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "1"]
"""
YOMON = """
FROM pytorch/pytorch:latest
WORKDIR /app
COPY . /app
RUN pip install fastapi uvicorn onnxruntime
RUN apt-get update && apt-get install -y curl
EXPOSE 8000
CMD uvicorn main:app --host 0.0.0.0 --port 8000
"""
# requirements.txt tarkibi (hajm bahosi uchun)
TALABLAR = {"yaxshi": ["fastapi", "uvicorn", "onnxruntime", "numpy", "pydantic"],
            "yomon": ["fastapi", "uvicorn", "onnxruntime", "numpy", "pydantic", "torch",
"torchvision"]}
# Taxminiy o'rnatilgan hajmlar, MB (pip show / du bilan o'lchanadi; CPU g'ildiraklar)
HAJM_MB = {"python:3.12-slim": 130, "python:3.12": 1020, "pytorch/pytorch:latest": 7200,
"nvidia/cuda:12.4.1-runtime-ubuntu22.04": 2900,
           "fastapi": 15, "uvicorn": 10, "onnxruntime": 60, "numpy": 35, "pydantic": 20,
"torch": 900, "torchvision": 25, "apt_curl_kesh": 45}
```

```
def tahlil(df_text):
    satrlar = [l.strip() for l in df_text.strip().splitlines() if l.strip() and not
l.startswith("#")]
    froms = [l for l in satrlar if l.upper().startswith("FROM ")]
    oxirgi_from = satrlar.index(froms[-1]) if froms else 0
    yakuniy = satrlar[oxirgi_from:]                      # faqat oxirgi bosqich tasvirga
kiradi
    asos = froms[-1].split()[1]
    q = {}
    q["Ko'p bosqichli qurish (>=2 FROM)"] = len(froms) >= 2
    q["Asos tasvir 'latest' emas (versiya qotirilgan)"] = not any(f.split()
[1].endswith(":latest") or ":" not in f.split()[1] for f in froms)
    q["Yengil asos tasvir (slim/runtime)"] = any(k in asos for k in ("slim", "runtime",
"alpine", "distroless"))
    q["USER root emas (yakuniy bosqichda)"] = any(l.upper().startswith("USER ") and l.split()
[1] != "root" for l in yakuniy)
    q["pip --no-cache-dir"] = all("--no-cache-dir" in l for l in satrlar if "pip install" in l)
    q["HEALTHCHECK bor"] = any(l.upper().startswith("HEALTHCHECK") for l in yakuniy)
    q["EXPOSE bor"] = any(l.upper().startswith("EXPOSE") for l in yakuniy)
    q["CMD exec (JSON) shaklida"] = any(l.startswith("CMD [") for l in yakuniy)
    q["Bog'liqliklar koddan OLDIN ko'chiriladi (kesh)"] = any("requirements" in l for l in
satrlar) and \
        next((i for i, l in enumerate(satrlar) if "requirements" in l), 99) < next((i for i, l
in enumerate(satrlar) if re.match(r"COPY (app/?|\.) ", l)), 0)
    q["apt keshi tozalangan"] = all("rm -rf /var/lib/apt/lists" in l for l in satrlar if "apt-
get install" in l)
    q["Sozlamalar ENV orqali (12 omil)"] = any(l.startswith("ENV ") and "MODEL_PATH" in l for l
in yakuniy)
    return q, asos

def hajm(asos, paketlar, apt_kesh=False):
    mb = HAJM_MB.get(asos, 500) + sum(HAJM_MB.get(p, 20) for p in paketlar) +
(HAJM_MB["apt_curl_kesh"] if apt_kesh else 0)
    return mb

print("=== Dockerfile tekshiruvi ===")
natija = {}
for nomi, text in (("yaxshi", YAXSHI), ("yomon", YOMON)):
    q, asos = tahlil(text); natija[nomi] = q
    ball = sum(q.values())
    print(f"\n[{nomi.upper()}] asos: {asos}   ball {ball}/{len(q)}")
    for k, v in q.items():
        print(f"  {'PASS' if v else 'FAIL'}  {k}")
jad = pd.DataFrame({"qoida": list(natija["yaxshi"]), "yaxshi": list(natija["yaxshi"].values()),
"yomon": list(natija["yomon"].values())})
print("\nQiyosiy jadval:"); print(jad.replace({True: "+", False: "-"}).to_string(index=False))

print("\n=== Tasvir hajmi bahosi (MB) ===")
rows = [dict(variant="yaxshi: slim + ORT (ko'p bosqichli)", MB=hajm("python:3.12-slim",
TALABLAR["yaxshi"])),
        dict(variant="o'rta: python:3.12 (to'liq) + ORT", MB=hajm("python:3.12",
TALABLAR["yaxshi"])),
        dict(variant="yomon: pytorch/pytorch + hamma narsa", MB=hajm("pytorch/pytorch:latest",
TALABLAR["yomon"], apt_kesh=True)),
        dict(variant="GPU: cuda-runtime + onnxruntime-gpu", MB=hajm("nvidia/cuda:12.4.1-
runtime-ubuntu22.04", TALABLAR["yaxshi"]) + 250)]
dh = pd.DataFrame(rows); dh["nisbat"] = (dh.MB / dh.MB[0]).round(1)
# Tortish vaqti 100 Mbit/s tarmoqda va 20 tugunli klasterga tarqatish
dh["tortish_s_100Mbit"] = (dh.MB * 8 / 100).round(0); dh["20_tugun_GB"] = (dh.MB * 20 /
1024).round(1)
print(dh.to_string(index=False))
print("\nXulosa: inferens uchun PyTorch kerak emas — ONNX Runtime yetarli; yengil asos + ko'p
bosqichli qurish tasvirni ~15x kichraytiradi,")
```

```
print("  bu K8s'da yangi Pod'ning ishga tushish vaqtini (tortish + start) va HPA reaksiyasini bevosita belgilaydi.")
print("Real labda: docker build -t digits-infer:1.0 . && docker images digits-infer && docker run --cpus=2 --memory=1g ...")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Dockerfile tekshiruvi ===`, `[YAXSHI]`, `[YOMON]`, `=== Tasvir hajmi bahosi (MB) ===`

TAJRIBA QILIB KO‘RING

- O'zingizning Dockerfile'ingizni `YAXSHI` o'rniga qo'yib tekshiring.
- Yangi qoida qo'shing: `COPY . .` taqiqlangan (.dockerignore'siz sirlar kiradi).
- GPU varianti uchun `nvidia/cuda:*-devel` asosini qo'shib hajm farqini ko'ring (devel ~2x runtime).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Kubernetes manifestlari: pyyaml bilan validatsiya va 18 qoida

MAQSAD

Deployment/ConfigMap/Service/HPA YAML'i `yaml.safe_load_all` bilan o'qilib 18 qoida bo'yicha tekshiriladi: replicas ≥ 2, selector/label mosligi, requests/limits, readiness va liveness farqi va vaqtlari, runAsNonRoot, RollingUpdate maxUnavailable 0, Service port/selector mosligi, HPA min/max/behavior/target, Prometheus annotatsiyalari. Tipik xatolar bilan yozilgan ikkinchi manifest 2/18 oladi. HPA chegarasi millicore'da hisoblanadi.

> **Kutilgan natija:** To'g'ri manifest 18/18 PASS, xatoli 2/18; requests 1500m × 60 % = 900m chegara; 2 nusxa 95 % da → 4 nusxa.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Kubernetes manifestlari: Deployment / ConfigMap / Service / HPA — pyyaml bilan
tahlil va qoidalar tekshiruvi
import yaml, pandas as pd

MANIFEST = """
apiVersion: apps/v1
kind: Deployment
metadata: { name: digits-infer, labels: { app: digits-infer } }
spec:
  replicas: 2
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
  selector: { matchLabels: { app: digits-infer } }
  template:
    metadata:
      labels: { app: digits-infer }
      annotations: { prometheus.io/scrape: "true", prometheus.io/port: "8000",
prometheus.io/path: "/metrics" }
    spec:
      securityContext: { runAsNonRoot: true, runAsUser: 10001 }
      containers:
        - name: infer
          image: digits-infer:1.0
          imagePullPolicy: IfNotPresent
          ports: [{ containerPort: 8000 }]
          env:
            - { name: MODEL_PATH, valueFrom: { configMapKeyRef: { name: digits-config, key:
model_path } } }
            - { name: ORT_THREADS, value: "2" }
          resources:
            requests: { cpu: "1500m", memory: "600Mi" }
            limits:   { cpu: "2",     memory: "1Gi" }
          readinessProbe: { httpGet: { path: /readyz, port: 8000 }, initialDelaySeconds: 5,
periodSeconds: 5, failureThreshold: 12 }
          livenessProbe:  { httpGet: { path: /healthz, port: 8000 }, initialDelaySeconds: 30,
periodSeconds: 15 }
          volumeMounts: [{ name: models, mountPath: /models, readOnly: true }]
      volumes:
        - name: models
          hostPath: { path: /data/models, type: Directory }
---
```

```
apiVersion: v1
kind: ConfigMap
metadata: { name: digits-config }
data: { model_path: /models/digits_int8.onnx }
---
apiVersion: v1
kind: Service
metadata: { name: digits-infer }
spec:
  type: NodePort
  selector: { app: digits-infer }
  ports: [{ port: 80, targetPort: 8000, nodePort: 30080 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: digits-infer }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: digits-infer }
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 60 } }
  behavior:
    scaleUp:   { stabilizationWindowSeconds: 30 }
    scaleDown: { stabilizationWindowSeconds: 180 }
"""

# "Yangi boshlovchi" manifesti — tipik xatolar bilan
XATOLI = """
apiVersion: apps/v1
kind: Deployment
metadata: { name: digits-infer }
spec:
  replicas: 1
  selector: { matchLabels: { app: digits-infer } }
  template:
    metadata: { labels: { app: digits } }
    spec:
      containers:
        - name: infer
          image: digits-infer:latest
          ports: [{ containerPort: 8000 }]
          livenessProbe: { httpGet: { path: /healthz, port: 8000 }, initialDelaySeconds: 2,
periodSeconds: 5 }
---
apiVersion: v1
kind: Service
metadata: { name: digits-infer }
spec:
  selector: { app: digits-infer }
  ports: [{ port: 80, targetPort: 9000 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: digits-infer }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: digits-infer }
  minReplicas: 1
  maxReplicas: 3
  metrics: [{ type: Resource, resource: { name: cpu, target: { type: Utilization,
averageUtilization: 60 } } }]
"""
```

```
def cpu_m(v):
    """CPU qiymatini millicore'ga: '1500m' -> 1500, '2' -> 2000."""
    v = str(v); return int(v[:-1]) if v.endswith("m") else int(float(v) * 1000)

def tekshir(text):
    obj = {d["kind"]: d for d in yaml.safe_load_all(text) if d}
    q = {}
    dep = obj.get("Deployment", {}); tpl = dep.get("spec", {}).get("template", {}); pod =
tpl.get("spec", {})
    k = pod.get("containers", [{}])[0]; res = k.get("resources", {})
    q["Deployment, Service, HPA, ConfigMap mavjud"] = all(x in obj for x in ("Deployment",
"Service", "HorizontalPodAutoscaler", "ConfigMap"))
    q["replicas >= 2 (bitta nuqta nosozligi yo'q)"] = dep.get("spec", {}).get("replicas", 0) >=
2
    q["selector == template labels"] = dep.get("spec", {}).get("selector",
{}).get("matchLabels") == tpl.get("metadata", {}).get("labels")
    q["image tegi 'latest' emas"] = ":" in k.get("image", "") and not
k["image"].endswith(":latest")
    q["resources.requests (CPU va xotira) — HPA uchun shart"] = all(x in res.get("requests",
{}) for x in ("cpu", "memory"))
    q["resources.limits (CPU va xotira)"] = all(x in res.get("limits", {}) for x in ("cpu",
"memory"))
    q["limits >= requests"] = bool(res.get("limits")) and cpu_m(res["limits"]["cpu"]) >=
cpu_m(res["requests"]["cpu"])
    q["readinessProbe bor"] = "readinessProbe" in k
    q["livenessProbe bor va readiness'dan kechroq boshlanadi"] = "livenessProbe" in k and \
        k["livenessProbe"].get("initialDelaySeconds", 0) >= k.get("readinessProbe",
{}).get("initialDelaySeconds", 0) + 10
    q["readiness va liveness yo'llari har xil"] = k.get("readinessProbe", {}).get("httpGet",
{}).get("path") != k.get("livenessProbe", {}).get("httpGet", {}).get("path")
    q["runAsNonRoot"] = pod.get("securityContext", {}).get("runAsNonRoot") is True
    q["RollingUpdate maxUnavailable: 0 (xizmat to'xtamaydi)"] = dep.get("spec",
{}).get("strategy", {}).get("rollingUpdate", {}).get("maxUnavailable") == 0
    svc = obj.get("Service", {}).get("spec", {})
    q["Service targetPort == containerPort"] = svc.get("ports", [{}])[0].get("targetPort") ==
k.get("ports", [{}])[0].get("containerPort")
    q["Service selector Pod labellariga mos"] = svc.get("selector") == tpl.get("metadata",
{}).get("labels")
    hpa = obj.get("HorizontalPodAutoscaler", {}).get("spec", {})
    q["HPA minReplicas >= 2 va max > min"] = hpa.get("minReplicas", 0) >= 2 and
hpa.get("maxReplicas", 0) > hpa.get("minReplicas", 0)
    q["HPA behavior (stabilizatsiya oynalari) bor"] = "behavior" in hpa
    q["HPA nishoni Deployment nomiga mos"] = hpa.get("scaleTargetRef", {}).get("name") ==
dep.get("metadata", {}).get("name")
    q["Prometheus scrape annotatsiyalari"] = tpl.get("metadata", {}).get("annotations",
{}).get("prometheus.io/scrape") == "true"
    return q, obj

print("=== Kubernetes manifest tekshiruvi ===")
jami = {}
for nomi, text in (("to'g'ri", MANIFEST), ("xatoli", XATOLI)):
    q, obj = tekshir(text); jami[nomi] = q
    print(f"\n[{nomi.upper()}] ob'ektlar: {list(obj)}   ball {sum(q.values())}/{len(q)}")
    for k, v in q.items():
        print(f"  {'PASS' if v else 'FAIL'}  {k}")

print("\nQiyosiy jadval:")
jad = pd.DataFrame({"qoida": list(jami["to'g'ri"]), "to'g'ri": list(jami["to'g'ri"].values()),
"xatoli": list(jami["xatoli"].values())})
print(jad.replace({True: "+", False: "-"}).to_string(index=False))

# HPA hisobi: foiz requests'ga nisbatan — chegara qancha millicore?
```

```
obj = {d["kind"]: d for d in yaml.safe_load_all(MANIFEST)}
req = cpu_m(obj["Deployment"]["spec"]["template"]["spec"]["containers"][0]["resources"]
["requests"]["cpu"])
hpa = obj["HorizontalPodAutoscaler"]["spec"]; target = hpa["metrics"][0]["resource"]["target"]
["averageUtilization"]
print(f"\n=== HPA hisobi ===\nrequests.cpu = {req}m, nishon {target} % -> masshtablash
chegarasi har Pod'da {req * target // 100}m CPU.")
print(f"Formula: kerakli_nusxa = ceil(joriy_nusxa x joriy_foiz / {target}). Masalan 2 nusxa 95
% da -> ceil(2 x 95/60) = {-(-2 * 95 // 60)} nusxa.")
print(f"Klaster sig'imi: maxReplicas {hpa['maxReplicas']} x limits 2 CPU = {hpa['maxReplicas']
* 2} vCPU kerak bo'lishi mumkin — tugunlar yetarlimi, tekshiring.")
print("\nReal labda: kubectl apply -f k8s/ && kubectl rollout status deployment/digits-infer &&
kubectl get hpa -w")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Kubernetes manifest tekshiruvi ===`, `[TO'G'RI]`, `[XATOLI]`, `=== HPA hisobi ===`

TAJRIBA QILIB KO‘RING

- GPU varianti uchun `limits: {nvidia.com/gpu: 1}` qo'shing va «GPU Pod'da HPA CPU bo'yicha ishlamaydi» qoidasini yozing.
- PodDisruptionBudget va Ingress ob'ektlarini qo'shib, ularga 2 tadan qoida yozing.
- `kubectl apply --dry-run=client` nima tekshiradi, sizning skriptingiz nima tekshiradi — farqni 3 jumlada yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. HPA avtomasshtablash simulyatsiyasi (navbat modeli)

MAQSAD

Minikube o'rniga diskret-vaqtli simulyatsiya: so'rov oqimi λ(t) (40 → 340 → 40 so'rov/s), har nusxa sig'imi μ = 1000/14 ms, CPU foizi = λ/(R·μ), HPA har 15 s da `ceil(R·cpu/nishon)` hisoblaydi, scaleUp/scaleDown stabilizatsiya oynalari va Pod start vaqti (20 s) hisobga olinadi, navbat va p95 kechikish Little qonuni bilan baholanadi. 7 ta konfiguratsiya (nishon 40/60/80 %, minReplicas 4, sekin start 60 s, maxReplicas 3, oynasiz) SLO buzilish vaqti va pod-soniya narxi bo'yicha solishtiriladi.

> **Kutilgan natija:** Asosiy konfiguratsiyada reaksiya ~65 s, SLO ~200 s buziladi; minReplicas 4 SLO buzilishini 4x kamaytiradi, narx +23 %; maxReplicas 3 — eng yomon.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. HPA avtomasshtablash simulyatsiyasi: so'rov oqimi -> CPU foizi -> nusxalar soni ->
navbat va kechikish
import math, numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

# --- Parametrlar (K8s manifestidan va 2-qadam o'lchovlaridan). O'zgartirib ko'ring!
XIZMAT_MS = 14.0            # bitta so'rovga xizmat vaqti (inferens + ishlov), ms  -> sig'im
1000/14 ~ 71 so'rov/s/nusxa
MU = 1000.0 / XIZMAT_MS
MIN_R, MAX_R = 2, 6         # HPA minReplicas / maxReplicas
NISHON = 60                 # averageUtilization, %
UP_OYNA, DOWN_OYNA = 30, 180   # stabilizationWindowSeconds
HPA_DAVR = 15               # HPA metrikani necha soniyada bir o'qiydi
START_S = 20                # yangi Pod tayyor bo'lgunicha (tasvir tortish + model yuklash +
readiness)
SLO_MS = 200                # p95 kechikish talabi
T = 900                     # simulyatsiya davomiyligi, s

def yuklama(t):
    """So'rov oqimi, so'rov/s: tinch -> keskin o'sish -> plato -> yana pasayish (video
kuzatuvda smena boshlanishi kabi)."""
    if t < 120: return 40
    if t < 180: return 40 + (t - 120) * 5          # 60 s ichida 40 -> 340
    if t < 480: return 340
    if t < 540: return 340 - (t - 480) * 5
    return 40

def simulyatsiya(min_r=MIN_R, max_r=MAX_R, nishon=NISHON, start_s=START_S, up_oyna=UP_OYNA,
down_oyna=DOWN_OYNA):
    tayyor, kutilayotgan = min_r, []            # kutilayotgan: [tayyor bo'lish vaqti, ...]
    navbat = 0.0; tavsiyalar = []; log = []
    for t in range(T):
        kutilayotgan_yangi = [x for x in kutilayotgan if x > t]
        tayyor += len(kutilayotgan) - len(kutilayotgan_yangi); kutilayotgan =
kutilayotgan_yangi
        lam = yuklama(t); sigim = tayyor * MU
        # Navbat dinamikasi (1 s qadam): kelgan - xizmat ko'rsatilgan
        navbat = max(0.0, navbat + lam - sigim)
        cpu = min(lam / sigim, 1.0) * 100 if sigim else 100     # foiz requests'ga nisbatan
(requests ~= limits deb olamiz)
        kutish_ms = navbat / sigim * 1000 if sigim else 1e4     # Little: navbatdagi so'rov
kutish vaqti = N / throughput
```

```
        p95_ms = XIZMAT_MS + kutish_ms + (XIZMAT_MS * 2 if cpu > 90 else 0)   # to'yinganda p95
keskin o'sadi (empirik qo'shimcha)
        # HPA boshqaruvchisi
        if t % HPA_DAVR == 0:
            istalgan = math.ceil(tayyor * cpu / nishon) if tayyor else min_r
            istalgan = min(max(istalgan, min_r), max_r)
            tavsiyalar.append((t, istalgan))
            jami = tayyor + len(kutilayotgan)
            if istalgan > jami:        # scaleUp: oxirgi up_oyna ichidagi tavsiyalarning ENG
KICHIGI (keskin sakrashlardan himoya)
                xavfsiz = min(v for tt, v in tavsiyalar if tt >= t - up_oyna)
                if xavfsiz > jami:
                    kutilayotgan += [t + start_s] * (xavfsiz - jami)
            elif istalgan < jami:      # scaleDown: oxirgi down_oyna ichidagi ENG KATTASI (erta
qisqartirishdan himoya)
                xavfsiz = max(v for tt, v in tavsiyalar if tt >= t - down_oyna)
                if xavfsiz < jami:
                    tayyor = max(min_r, xavfsiz)
        log.append(dict(t=t, lam=lam, tayyor=tayyor, cpu=cpu, navbat=navbat, p95_ms=p95_ms))
    df = pd.DataFrame(log)
    return df, dict(max_nusxa=int(df.tayyor.max()), max_p95_ms=round(df.p95_ms.max()),
SLO_buzilgan_s=int((df.p95_ms > SLO_MS).sum()),
                    max_navbat=int(df.navbat.max()), pod_soniya=int(df.tayyor.sum()),
                    reaksiya_s=int(df[df.tayyor > min_r].t.min() - 120) if (df.tayyor >
min_r).any() else None)

print("=== HPA simulyatsiyasi ===")
print(f"Nusxa sig'imi: {MU:.0f} so'rov/s; cho'qqi yuklama 340 so'rov/s -> kamida {math.ceil(340
/ MU / (NISHON / 100))} nusxa kerak ({NISHON} % nishonda)")
asosiy, x = simulyatsiya()
print("\nAsosiy konfiguratsiya (min 2, max 6, nishon 60 %, start 20 s):")
for k, v in x.items(): print(f"  {k}: {v}")
print("\nVaqt bo'yicha (har 60 s):")
print(asosiy[asosiy.t % 60 == 0][["t", "lam", "tayyor", "cpu", "navbat",
"p95_ms"]].round(0).to_string(index=False))

# Konfiguratsiyalarni solishtirish
print("\n=== Konfiguratsiyalar taqqosi ===")
rows = []
for nomi, kw in (("asosiy: min2 max6 nishon60 start20", {}),
                 ("nishon 80 % (tejamkor)", dict(nishon=80)),
                 ("nishon 40 % (zaxirali)", dict(nishon=40)),
                 ("minReplicas 4 (oldindan zaxira)", dict(min_r=4)),
                 ("sekin start 60 s (GPU Pod, TensorRT)", dict(start_s=60)),
                 ("maxReplicas 3 (klaster kichik)", dict(max_r=3)),
                 ("scaleUp oynasi 0 s (tez reaksiya)", dict(up_oyna=0))):
    _, r = simulyatsiya(**kw); rows.append(dict(konfiguratsiya=nomi, **r))
dk = pd.DataFrame(rows); dk["narx_nisbati"] = (dk.pod_soniya / dk.pod_soniya[0]).round(2)
print(dk.to_string(index=False))
print("\nXulosa: HPA reaktiv — yuklama o'sgach, metrika davri + stabilizatsiya + Pod start
vaqti o'tgachgina sig'im paydo bo'ladi;")
print("  shu oraliqda navbat to'planib p95 SLO buziladi. Yechimlar: minReplicas zaxirasi (narx
oshadi), pastroq nishon, tez start (kichik tasvir),")
print("  yoki so'rov/s bo'yicha prognozli masshtablash (KEDA). GPU Pod'larda start 30-60 s —
muammo yanada keskin.")

fig, ax = plt.subplots(3, 1, figsize=(9, 7.5), sharex=True)
ax[0].plot(asosiy.t, asosiy.lam, label="so'rov/s"); ax[0].plot(asosiy.t, asosiy.tayyor * MU, "-
-", label="sig'im (nusxa x mu)"); ax[0].set_ylabel("so'rov/s"); ax[0].legend();
ax[0].grid(alpha=.3)
ax[1].step(asosiy.t, asosiy.tayyor, where="post", color="tab:green", label="tayyor nusxalar");
ax[1].set_ylabel("nusxa"); ax[1].set_ylim(0, MAX_R + 1)
ax1b = ax[1].twinx(); ax1b.plot(asosiy.t, asosiy.cpu, color="tab:gray", alpha=.6, label="CPU
```

```
%"); ax1b.axhline(NISHON, color="tab:gray", ls=":"); ax1b.set_ylabel("CPU %");
ax[1].legend(loc="upper left"); ax[1].grid(alpha=.3)
ax[2].plot(asosiy.t, asosiy.p95_ms, color="tab:red"); ax[2].axhline(SLO_MS, color="k", ls="--",
label=f"SLO {SLO_MS} ms"); ax[2].set_yscale("log"); ax[2].set_ylabel("p95, ms");
ax[2].set_xlabel("vaqt, s"); ax[2].legend(); ax[2].grid(alpha=.3)
fig.suptitle("HPA simulyatsiyasi: yuklama, nusxalar, kechikish"); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== HPA simulyatsiyasi ===`, `=== Konfiguratsiyalar taqqosi ===`, `narx_nisbati`

TAJRIBA QILIB KO‘RING

- `yuklama(t)` ni sinusoidal (kunlik sikl) qiling — HPA qanday kuzatadi?
- So'rov/s bo'yicha masshtablash (KEDA) qo'shing: istalgan = ceil(λ/(μ·0,6)) — CPU bilan solishtiring.
- Navbat chegarasi qo'shing (masalan 500): ortiqcha so'rovlar 503 bilan rad etilsin va rad etilganlar sonini hisoblang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Cloud vs Edge vs Gibrid: kechikish, trafik, energiya va TCO kalkulyatori

MAQSAD

A1 SHV ssenariysi (16 kamera, 160 inferens/s, 3 yil) uchun uch arxitektura hisoblanadi: uchdan-uchga kechikish (RTT + navbat + inferens), tarmoq trafigi, mJ/inferens (PUE bilan), TCO (GPU soati, egress, qo'shimcha xizmatlar; Jetson narxi, energiya, texnik xizmat, zaxira). TCO 1/2/3/5 yil, sezgirlik tahlili (kameralar x2/x8, bulut narxi −30 %, hodisa bo'yicha oqim, xizmat narxi, RTT) va kesishish nuqtasi. Jetson/TPU yo'q — bu simulyatsiya, raqamlar o'zgartiriladigan parametrlar.

> **Kutilgan natija:** Cloud TCO Edge'dan 30–50 marta qimmat, kechikish ~6 marta katta; bulut narxi −30 % ham nisbatni saqlaydi; grafik: TCO yillar bo'yicha va sezgirlik ustunlari.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Cloud vs Edge vs Gibrid: kechikish, trafik, energiya va TCO kalkulyatori (A1 SHV
ssenariysi) + sezgirlik tahlili
import math, numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

# --- Ssenariy va narxlar (ochiq narxlar, taxminiy; hisobotda sana va manba ko'rsatiladi).
Hammasini o'zgartirib ko'ring.
S = dict(
    kameralar=16, fps_har_kamera=10,          # 16 kamera x 10 kadr/s = 160 inferens/s
    yillar=3, kvt_soat_usd=0.08, pue_cloud=1.3,
    # Cloud (T4 GPU, K8s)
    gpu_soat_usd=0.50, gpu_sigim_inf_s=380, zaxira_nusxa=2, rtt_ms=80, bulut_qoshimcha_foiz=30,
# egress, LB, loglar, monitoring
    oqim_mbit_kamera=4.0, egress_usd_gb=0.09, gpu_quvvat_w=70, inferens_ms_cloud=3.0,
navbat_ms_cloud=25,
    # Edge (Jetson Orin NX)
    edge_qurilma_usd=700, edge_korpus_usd=300, edge_quvvat_w=25, edge_sigim_inf_s=380,
edge_inferens_ms=19,
    edge_xizmat_usd_yil=150, edge_zaxira=1,     # texnik xizmat + bitta zaxira qurilma
    # Gibrid: chekkada hamma, bulutga shubhali kadrlar ulushi
    shubhali_ulush=0.05,
)

def hisob(S):
    inf_s = S["kameralar"] * S["fps_har_kamera"]
    soat = S["yillar"] * 365 * 24
    # ---- Cloud
    gpu_n = max(S["zaxira_nusxa"], math.ceil(inf_s / S["gpu_sigim_inf_s"]))
    gpu_usd = gpu_n * S["gpu_soat_usd"] * soat
    trafik_mbit = S["kameralar"] * S["oqim_mbit_kamera"]                                  #
uzluksiz oqim
    egress_gb = trafik_mbit / 8 / 1024 * 3600 * soat
    cloud_usd = (gpu_usd + egress_gb * S["egress_usd_gb"]) * (1 + S["bulut_qoshimcha_foiz"] /
100)
    cloud_lat = S["rtt_ms"] + S["navbat_ms_cloud"] + S["inferens_ms_cloud"]
    cloud_mj = S["gpu_quvvat_w"] * gpu_n / inf_s * 1000 * S["pue_cloud"]                  #
to'liq (GPU band bo'lmasa ham ishlaydi)
    # ---- Edge
    edge_n = math.ceil(inf_s / S["edge_sigim_inf_s"])
    kwh = S["edge_quvvat_w"] * edge_n * soat / 1000
```

```
    edge_usd = (edge_n + S["edge_zaxira"]) * (S["edge_qurilma_usd"] + S["edge_korpus_usd"]) +
kwh * S["kvt_soat_usd"] + S["edge_xizmat_usd_yil"] * S["yillar"] * edge_n
    edge_lat = S["edge_inferens_ms"]
    edge_mj = S["edge_quvvat_w"] * edge_n / inf_s * 1000
    # ---- Gibrid: Edge + bulutda faqat shubhali kadrlar (kichik GPU ulushi, egress kam)
    sh_inf_s = inf_s * S["shubhali_ulush"]
    gib_gpu_usd = 1 * S["gpu_soat_usd"] * soat * max(S["shubhali_ulush"] * 4, 0.1)        # 1
GPU qisman band (serverless/spot taxmini)
    gib_egress_gb = egress_gb * S["shubhali_ulush"] / 10                                   #
faqat kesilgan kadrlar
    gib_usd = edge_usd + (gib_gpu_usd + gib_egress_gb * S["egress_usd_gb"]) * (1 +
S["bulut_qoshimcha_foiz"] / 100)
    return pd.DataFrame([
        dict(arxitektura="Cloud (T4, K8s)", kechikish_p95_ms=cloud_lat,
trafik_mbit_s=round(trafik_mbit, 1), TCO_usd=round(cloud_usd),
             mJ_inferens=round(cloud_mj), oflayn="yo'q", maxfiylik="video chiqadi", nusxa=f"
{gpu_n} GPU"),
        dict(arxitektura="Edge (Jetson Orin NX)", kechikish_p95_ms=edge_lat,
trafik_mbit_s=0.01, TCO_usd=round(edge_usd),
             mJ_inferens=round(edge_mj), oflayn="ha", maxfiylik="video binoda", nusxa=f"
{edge_n}+{S['edge_zaxira']} qurilma"),
        dict(arxitektura="Gibrid (Edge + bulut tekshiruv)", kechikish_p95_ms=f"{edge_lat} /
{cloud_lat}", trafik_mbit_s=round(trafik_mbit * S["shubhali_ulush"] / 10, 2),
             TCO_usd=round(gib_usd), mJ_inferens=round(edge_mj * 1.1), oflayn="asosiy ha",
maxfiylik="faqat kadrlar", nusxa=f"{edge_n} qurilma + 1 GPU qisman"),
    ])

print("=== Cloud vs Edge ===")
print(f"Ssenariy: {S['kameralar']} kamera x {S['fps_har_kamera']} kadr/s = {S['kameralar'] *
S['fps_har_kamera']} inferens/s, {S['yillar']} yil")
df = hisob(S); print(df.to_string(index=False))
nisbat = df.TCO_usd[0] / df.TCO_usd[1]
print(f"\nCloud TCO Edge'dan {nisbat:.1f} marta qimmat; kechikish {df.kechikish_p95_ms[0] /
df.kechikish_p95_ms[1]:.1f} marta katta.")
print("Cloud mJ/inferens katta, chunki 2 ta zaxira GPU 160 inferens/s da kam band (sig'im
760/s) — bo'sh quvvat ham hisobga kiradi (PUE 1,3 bilan).")

# --- TCO yillar bo'yicha (1, 3, 5): Edge kapital xarajat oldinda, Cloud chiziqli o'sadi
print("\n=== TCO yillar bo'yicha (USD) ===")
rows = []
for y in (1, 2, 3, 5):
    d = hisob({**S, "yillar": y}); rows.append(dict(yil=y, cloud=d.TCO_usd[0],
edge=d.TCO_usd[1], gibrid=d.TCO_usd[2]))
dy = pd.DataFrame(rows); print(dy.to_string(index=False))

# --- Sezgirlik tahlili
print("\n=== Sezgirlik tahlili (3 yil, TCO USD) ===")
rows = []
for nomi, kw in (("asosiy", {}), ("kameralar x2", dict(kameralar=32)), ("kameralar x8 (128)",
dict(kameralar=128)),
                 ("bulut narxi -30 %", dict(gpu_soat_usd=0.35, egress_usd_gb=0.063)), ("hodisa
bo'yicha oqim (0.5 Mbit/kamera)", dict(oqim_mbit_kamera=0.5)),
                 ("Edge xizmat 500 $/yil", dict(edge_xizmat_usd_yil=500)), ("RTT 150 ms (uzoq
mintaqa)", dict(rtt_ms=150))):
    d = hisob({**S, **kw}); rows.append(dict(holat=nomi, cloud=d.TCO_usd[0], edge=d.TCO_usd[1],
gibrid=d.TCO_usd[2],
                                             cloud_edge=round(d.TCO_usd[0] / d.TCO_usd[1], 1),
cloud_p95=d.kechikish_p95_ms[0]))
ds = pd.DataFrame(rows); print(ds.to_string(index=False))

# Qachon Cloud arzonroq? Kameralar soni juda kam bo'lsa (bitta kamera uchun ham Jetson sotib
olinadi) yoki yuklama notekis bo'lsa
print("\n=== Kesishish nuqtasi ===")
```

```
for kam in (1, 2, 4, 8):
    d = hisob({**S, "kameralar": kam, "zaxira_nusxa": 1, "edge_zaxira": 0})
    print(f"  {kam} kamera, zaxirasiz: cloud {d.TCO_usd[0]:>7} $   edge {d.TCO_usd[1]:>6} $   -
> {'Cloud' if d.TCO_usd[0] < d.TCO_usd[1] else 'Edge'} arzon")
print("Xulosa: SHV ssenariysida (uzluksiz 160 inferens/s, maxfiy video) Edge aniq ustun; Cloud
10 000 foydalanuvchili mobil ilova kabi")
print("  notekis, geografik tarqoq yuklamada oqilona. Gibrid: chekkada 95 % qaror, bulutda
sifat nazorati va qayta o'qitish uchun ma'lumot.")
print("Eslatma: bu simulyatsiya — Jetson/TPU/GPU o'lchovlari emas; real labda raqamlar
tegrastats, nvidia-smi va bulut hisob-fakturasidan olinadi.")

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(dy.yil, dy.cloud, "o-", label="Cloud"); ax[0].plot(dy.yil, dy.edge, "s-",
label="Edge"); ax[0].plot(dy.yil, dy.gibrid, "^-", label="Gibrid")
ax[0].set_xlabel("yil"); ax[0].set_ylabel("TCO, USD"); ax[0].set_title("TCO yillar bo'yicha");
ax[0].legend(); ax[0].grid(alpha=.3)
xx = np.arange(len(ds)); w = 0.27
ax[1].bar(xx - w, ds.cloud, w, label="Cloud"); ax[1].bar(xx, ds.edge, w, label="Edge");
ax[1].bar(xx + w, ds.gibrid, w, label="Gibrid")
ax[1].set_xticks(xx); ax[1].set_xticklabels(ds.holat, rotation=25, ha="right", fontsize=7);
ax[1].set_yscale("log"); ax[1].set_ylabel("TCO 3 yil, USD (log)")
ax[1].set_title("Sezgirlik tahlili"); ax[1].legend(); ax[1].grid(axis="y", alpha=.3)
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Cloud vs Edge ===`, `=== TCO yillar bo'yicha (USD) ===`, `=== Sezgirlik tahlili`, `=== Kesishish nuqtasi ===`

TAJRIBA QILIB KO‘RING

- Yuklama notekis (kuniga 2 soat cho'qqi) ssenariysini qo'shing: bulutda GPU faqat ishlaganda to'lansin — kim yutadi?
- Muhandis vaqti (OTA, flot boshqaruvi) xarajatini Edge'ga 1000 qurilma uchun qo'shing.
- Ma'lumotlar markazi PUE 1,1 va 1,5 bilan energiya ustunini qayta hisoblang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Probe'lar ssenariysi** 1-qadam xizmatiga `/readyz` ni vaqtincha 503 ga o'tkazuvchi (model qayta yuklanmoqda) `/admin/reload` endpointi qo'shing. TestClient bilan ketma-ketlikni ko'rsating: ready → reload → 503 → ready. Har holatda K8s nima qilishini (trafik, qayta ishga tushirish) 1 jumlada izohlang.	oson	5
2	**Dockerfile qoidalarini kengaytirish** 3-qadam tahlilchisiga 3 ta yangi qoida qo'shing (masalan: `COPY . .` taqiqi, `ADD` o'rniga `COPY`, `WORKDIR` mavjudligi, `pip` versiyasi qotirilgan `==`). Ikkala Dockerfile'ni qayta baholang va qoidalar jadvalini topshiring.	oson	5
3	**TCO parametrlarini yangilash** 6-qadam kalkulyatoridagi GPU soati, egress va Jetson narxini bugungi ochiq narxlar bilan (manba va sana ko'rsatib) almashtiring, 3 yillik TCO jadvalini qayta hisoblang va nisbat qanday o'zgarganini 2–3 jumlada tushuntiring.	oson	5
4	**Kechikish byudjeti dekompozitsiyasi** 2-qadamda to'liq so'rov kechikishini qismlarga ajrating: JSON parse, pydantic validatsiya, oldindan ishlov, inferens, javob serializatsiyasi (har qismga `perf_counter` qo'ying). c=1 va c=8 uchun qismlar ulushini ustunli diagramma bilan ko'rsating va eng katta qismni kamaytirish usulini (masalan, `orjson`, raw bytes kirish) sinab natija bering.	o'rta	10
5	**Manifest validatori kengaytmasi** 4-qadam tekshiruvchisiga PodDisruptionBudget, Ingress (TLS) va NetworkPolicy ob'ektlarini qo'shing, ularga kamida 6 ta yangi qoida yozing (masalan, PDB minAvailable ≥ 1, Ingress TLS secret mavjud, NetworkPolicy faqat 8000-portga ruxsat). Jami 24+ qoidali hisobot va «to'g'ri» manifestni 100 % PASS holatiga keltiring.	o'rta	10
6	**KEDA uslubida so'rov/s bo'yicha masshtablash** 5-qadam simulyatsiyasiga ikkinchi masshtablash siyosati qo'shing: istalgan = ceil(λ/(μ·0,6)) (so'rov/s bo'yicha, CPU emas). CPU-HPA bilan bir xil yuklama profilida SLO buzilish vaqti, maksimal navbat va pod-soniya narxini solishtiring; GPU xizmatlari uchun qaysi biri to'g'ri ekanini asoslang.	o'rta	10
7	**Rolling update simulyatsiyasi** 5-qadam modeliga yangilanish (rollout) qo'shing: t=400 s da `maxSurge=1, maxUnavailable=0` bilan nusxalar birma-bir almashtiriladi (yangi Pod 20 s da tayyor, keyin eski o'chadi). `maxUnavailable=1` varianti bilan solishtiring: yangilanish davomida sig'im, navbat va SLO buzilishi grafigi; GPU'lar soni cheklangan (2 GPU) holatda qaysi strategiya ishlashini tushuntiring.	o'rta	10
8	**Chekka qurilma simulyatori va gibrid mezon** 1-qadam xizmatining «Edge» nusxasini yarating: ORT 1 oqim, har so'rovga ARM CPU oldindan ishlovini taqlid qiluvchi 15 ms kechikish, 2 Vt quvvat modeli. «Cloud» nusxasi: 2 oqim + 80 ms RTT. Gibrid marshrutlash qoidasini yozing (ishonch <	qiyin	15

	chegara bo'lsa bulutga yuboriladi) va chegarani 0,5–0,99 oralig'ida o'zgartirib, bulutga ketgan so'rovlar ulushi, o'rtacha kechikish, umumiy aniqlik va trafikni grafikda ko'rsating; optimal chegarani asoslang.		
9	**Xizmatga nosozlik kiritish (chaos) sinovi** 2-qadam yuklama sinoviga nosozlik ssenariylari qo'shing: (a) 10 % so'rovlarda sun'iy 500 ms kechikish, (b) 5 % so'rovlarda 500 xato, (c) t=5 s da model qayta yuklash (readyz 503, 2 s). Har ssenariy uchun p50/p95/p99, xatolar ulushi va `/metrics` dagi ko'rsatkichlarni jadvalga yig'ing; mijoz tomonida timeout + 1 marta qayta urinish (retry) strategiyasini qo'shib natijani solishtiring. 4–5 betlik hisobot.	qiyin	15
10	**To'liq joylashtirish hisoboti o'z ssenariyingiz uchun** A1 da tanlagan tizimingiz uchun 1–6 qadamlarni to'liq takrorlang: o'z ONNX modelingiz bilan xizmat (probe'lar, metrikalar), yuklama sinovi (kamida 5 parallellik darajasi), o'z Dockerfile va K8s manifestlaringizni skriptlar bilan 100 % PASS holatiga keltirish, o'z yuklama profilingiz bilan HPA simulyatsiyasi (kamida 3 konfiguratsiya), Cloud/Edge/Gibrid TCO 1/3/5 yil va sezgirlik tahlili. 6–8 betlik hisobot: yakuniy arxitektura tavsiyasi va A1 strukturaviy sxemasining yangilangan varianti bilan.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: rentgen tasvirlarini triaj qilish xizmati — probe'lar, yuklama va kasalxona ichidagi Edge o'rta

Soha: Tibbiyot (radiologiya)

Viloyat kasalxonasi rentgen tasvirlarini «shoshilinch/odatiy» deb saralaydigan xizmatni ishga tushirmoqda; tasvirlar surrogati — digits 8x8 CNN. Ertalab 8:00-10:00 da yuklama 5 so'rov/s dan 60 so'rov/s ga ko'tariladi, qolgan vaqtda 5 so'rov/s. Bemor ma'lumotlari binodan chiqmasligi kerak, shuning uchun bulut varianti faqat anonimlashtirilgan tasvirlar bilan ko'rib chiqiladi. Siz FastAPI xizmatini yozib TestClient bilan sinaysiz, yuklama sinovi o'tkazasiz, Dockerfile va K8s manifestlarini tekshirasiz, HPA ni ertalabki profil bilan simulyatsiya qilasiz va kasalxona serveri (Edge) bilan bulutni TCO va maxfiylik bo'yicha solishtirasiz.

Model	kichik CNN (A3 s1) -> ONNX, 2 sinf (shoshilinch = raqamlar 1,4,7)
Xizmat turi	tasvir tasnifi, /predict 64 piksel
SLO	p95 <= 100 ms, mavjudlik 99,9 %
Yuklama profili	5 -> 60 -> 5 so'rov/s (ertalabki cho'qqi 2 soat, o'sish 10 daqiqa)
Xizmat vaqti (sim)	25 ms
HPA	min 2, max 5, nishon 55 %, start 25 s
TCO	3 yil, kasalxona serveri 2500 USD + 40 Vt vs bulut GPU 0,50 USD/soat, RTT 60 ms

QADAMLAR

1. Laboratoriya s1 kabi xizmatni yozing: /healthz, /readyz, /predict (tasvir -> sinf + ishonch + timing), /metrics; readyz yuklanishdan oldin 503 ekanini TestClient bilan ko'rsating; 200 tasvirda aniqlik.
2. Laboratoriya s2 kabi yuklama sinovi: ketma-ket 300 so'rov (sof inferens ulushi), parallel 1/2/4/8/16 mijoz jadvali (so'rov/s, p50/p95/p99, little_N), SLO p95 <= 100 ms tekshiruvi.
3. Dockerfile tahlili (s3): o'z Dockerfile'ingizni yozing (slim + ORT, root'siz, HEALTHCHECK /readyz) va 11 qoidadan o'tkazing; tasvir hajmi bahosi va kasalxona 100 Mbit/s tarmog'ida tortish vaqti.
4. K8s manifestlari (s4): Deployment/ConfigMap/Service/HPA (min 2, max 5, nishon 55 %) 18 qoidadan 100 % o'tsin; HPA chegarasini millicore'da hisoblang.
5. HPA simulyatsiyasi (s5): yuklama(t) ni ertalabki profilga o'zgartiring (T = 3 soat, 10 daqiqada 5 -> 60), xizmat vaqti 25 ms; asosiy + 3 konfiguratsiya (nishon 40 %, min 3, start 60 s) uchun SLO buzilish vaqti va pod-soniya.
6. TCO (s6): kasalxona serveri (Edge) vs bulut vs gibrid (faqat anonim shubhali tasvirlar bulutga) 1/3/5 yil; kechikish (RTT 60 ms) va maxfiylik ustunlari; xulosa va tavsiya.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Dockerfile va K8s YAML (PASS natijasi bilan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

2-variant. Chakana savdo: mahsulot tavsiya xizmati — top-5, Qora juma yuklamasi va bulut tanlovi o'rta

Soha: Chakana savdo

Onlayn do'kon har sahifa ochilganda foydalanuvchiga 5 ta tavsiya ko'rsatadi; model — foydalanuvchi va mahsulot embeddinglari ustidagi MLP (A3 kabi, sintetik). Odatiy yuklama 150 so'rov/s, Qora juma kuni 20 daqiqa ichida 3000 so'rov/s ga chiqadi va 6 soat davom etadi. Har so'rov 200 nomzodni baholaydi, tavsiya 80 ms ichida kelmasa sahifa tavsiyasiz ochiladi (daromad yo'qoladi). Siz /recommend endpointini yozasiz, yuklama sinovida nomzodlar sonining kechikishga ta'sirini o'lchaysiz, manifestlarni tekshirasiz, HPA ni Qora juma profilida simulyatsiya qilasiz va faqat bulut variantlari (doimiy nusxa vs avtomasshtab vs oldindan rejalashtirilgan masshtab) TCO sini solishtirasiz.

Model	Embedding(20000,16)+Embedding(5000,16)+MLP 32-64-1 -> ONNX
Xizmat turi	tavsiya, /recommend {user_id, item_ids[200]} -> top-5
SLO	p95 <= 80 ms
Yuklama profili	150 -> 3000 so'rov/s (20 daqiqada), 6 soat plato, keyin 150
Xizmat vaqti (sim)	6 ms (200 nomzod)
HPA	min 3, max 40, nishon 65 %, start 30 s
Bulut narxi	vCPU-soat 0,045 USD, nusxa 2 vCPU

QADAMLAR

1. A3 dagi embedding+MLP modelini (sintetik) o'qitib ONNX'ga eksport qiling; FastAPI xizmati: /readyz, /recommend (200 nomzodni paketda baholab top-5 qaytaradi), /metrics (so'rovlar, p95, nomzodlar soni gistogrammasi).
2. Yuklama sinovi: nomzodlar soni 20/50/100/200/500 uchun ketma-ket p50/p95; keyin 200 nomzod bilan parallel 1/2/4/8/16 mijoz jadvali va SLO tekshiruvi.
3. Dockerfile (s3) 11 qoida va K8s manifest (s4) 18 qoida — o'z fayllaringiz 100 % PASS; HPA max 40 uchun klaster sig'imi (vCPU) hisobi.
4. HPA simulyatsiyasi (s5): Qora juma profili (T = 8 soat, 1 s qadam yoki 5 s qadam), xizmat vaqti 6 ms, min 3 / max 40; SLO buzilish vaqti, maksimal navbat, pod-soniya.
5. Uch siyosat taqqosi: (a) CPU-HPA, (b) oldindan rejalashtirilgan masshtab (t = -30 daqiqada 30 nusxaga), (c) doimiy 40 nusxa; har biri uchun SLO buzilgan soniyalar va vCPU-soat narxi.
6. TCO (s6) ni bulut ichida qayta yozing: 3 siyosat uchun yillik narx (oddiy kunlar + 5 ta aksiya kuni), yo'qotilgan tavsiyalar soni (SLO buzilgan vaqt x so'rov/s) va har yo'qotilgan tavsiya 0,02 USD deb daromad yo'qotishini hisoblang; xulosa.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Uch masshtablash siyosati taqqos jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

3-variant. Sanoat: vibratsiya anomaliyasini baholash xizmati — sex shlyuzida Edge va ARM tasviri o'rta

Soha: Sanoat

Zavodda 200 ta dvigatel sensorlari har soniyada bittadan 256 nuqtalik oyna yuboradi (jami 200 so'rov/s, doimiy). Autoenkoder (A3 kabi) rekonstruksiya xatosini hisoblab anomaliya ballini qaytaradi; qaror 500 ms ichida kerak, zavod tarmog'i internetga 10 Mbit/s kanal orqali ulangan va tez-tez uziladi. Siz /score xizmatini yozasiz, yuklamani sinaysiz, ARM shlyuzi uchun Dockerfile (python:3.12-slim arm64, ORT) tahlil qilasiz, K8s o'rniga bitta tugunli k3s manifestini tekshirasiz, HPA simulyatsiyasida sensorlar sonining o'sishini (200 -> 600) modellashtirasiz va Edge shlyuz vs bulut TCO ni oflayn ishlash talabi bilan solishtirasiz.

Model	Autoenkoder 256-64-16-64-256 -> ONNX (anomaliya = MSE > chegara)
Xizmat turi	anomaliya balli, /score {signal[256]} -> {score, is_anomaly}
SLO	p95 <= 500 ms, oflayn ishlash majburiy
Yuklama profili	200 so'rov/s doimiy; 2-yilda 600
Xizmat vaqti (sim)	4 ms
Shlyuz	ARM 4 yadro, 4 GB (Edge), 450 USD + 12 Vt
Bulut	2 vCPU nusxa 0,09 USD/soat, egress 0,09 USD/GB, RTT 90 ms
Trafik	256 x 4 bayt x 200/s

QADAMLAR

1. Autoenkoderni sintetik vibratsiyada o'qiting (A3 4-variant usuli), chegarani 99-foizil qilib ONNX'ga eksport qiling; FastAPI /score, /readyz, /metrics (anomaliya ulushi gauge bilan).
2. Yuklama sinovi: parallel 1/2/4/8/16 mijoz; 200 so'rov/s talabini qaysi parallellikda bajarish mumkin; little_N; /score_batch (50 oyna bitta so'rovda) bilan solishtiring.
3. Dockerfile: arm64 uchun slim + onnxruntime (torch'siz) variant va «hamma narsa bilan» variant; 11 qoida va hajm bahosi; 10 Mbit/s kanalda tasvirni shlyuzga yetkazish vaqti.
4. k3s manifesti (bitta tugun): Deployment replicas 2 (ikki jarayon), Service, HPA max 3; 18 qoidadan o'ting, bitta tugunda replicas >= 2 nimani beradi va nimani bermaydi — izohlang.
5. HPA simulyatsiyasi: yuklama 200 -> 600 so'rov/s 1 yil davomida chiziqli o'sadi deb (simulyatsiyada 600 s ga siqilgan), xizmat vaqti 4 ms; shlyuz sig'imi qachon tugaydi, ikkinchi shlyuz qachon kerak.
6. TCO (s6): Edge shlyuz (1+1 zaxira) vs bulut vs gibrid (shlyuzda ball, bulutga faqat anomal oynalar) 1/3/5 yil; trafik GB/oy; oflayn uzilishda (yiliga 40 soat) bulut variantida nechta oyna yo'qoladi; xulosa.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Ikki Dockerfile taqqos jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

4-variant. Telekom: SMS spam filtri xizmati — doimiy 2000 so'rov/s, paketli endpoint va K8s manifest oson

Soha: Telekom

Mobil operator SMS markazidan o'tayotgan xabarlarni spam/oddiy deb belgilaydi: doimiy oqim 2000 xabar/s, kechikish talabi yumshoq (p95 300 ms), lekin xizmat to'xtamasligi kerak. Model — 1000 o'lchamli so'zlar sumkasi MLP (A3 15-variant kabi, sintetik). SMS markazi xabarlarni 100 talik paketlarda yuborishi mumkin. Siz /classify va /classify_batch endpointlarini yozasiz, yuklama sinovida yakka va paketli rejimni solishtirasiz, Dockerfile va K8s manifestlarini qoidalardan o'tkazasiz va paketli rejimda nechta nusxa kerakligini Little qonuni bilan hisoblaysiz.

Model	MLP 1000-256-2 -> ONNX
Xizmat turi	matn tasnifi, /classify {vector[1000]} va /classify_batch [100 ta]
SLO	p95 <= 300 ms, mavjudlik 99,95 %
Yuklama profili	2000 xabar/s doimiy
Paket	1 va 100
K8s	replicas 3, HPA min 3 max 8, nishon 60 %, PDB minAvailable 2
Nusxa	2 vCPU, 1 GiB

QADAMLAR

1. Sintetik so'zlar sumkasi ma'lumotida MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI xizmati /readyz, /classify, /classify_batch, /metrics (xabarlar soni, spam ulushi).
2. Yuklama sinovi (s2): yakka so'rov parallel 1/2/4/8/16 mijoz; paketli (100 ta) so'rov parallel 1/2/4 mijoz; xabar/s ko'rsatkichini ikkala rejim uchun jadvalga yozing.
3. Little qonuni: 2000 xabar/s uchun yakka va paketli rejimda nusxalar soni (o'lchangan throughput/nusxa asosida); paketli rejimda 100 ta xabar yig'ilish kutishi (100/2000 s) kechikishga qo'shiladi — SLO ga sig'adimi.
4. Dockerfile (s3): o'z variantingiz 11 qoidadan o'tsin; tasvir hajmi va 8 tugunga tarqatish GB.
5. K8s manifest (s4): Deployment replicas 3, PDB (minAvailable 2) qo'shing va unga 2 ta yangi qoida yozing (jami 20); HPA chegarasi millicore'da.
6. Xulosa: operatorga paketli yoki yakka rejim, nechta nusxa, PDB nega kerak (tugun yangilanishida xizmat to'xtamasligi), GPU kerakmi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); K8s YAML (PDB bilan) va 20 qoidali tekshiruv natijasi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

5-variant. Energetika: aqlli hisoblagichlar uchun iste'mol prognozi — 15 daqiqalik burst, HPA va batch-job taqqosi qiyin

Soha: Energetika

Elektr tarmog'i operatori 40 000 ta aqlli hisoblagichdan har 15 daqiqada ko'rsatkich oladi va har biri uchun keyingi soat prognozini hisoblaydi; so'rovlar 15 daqiqalik oynaning birinchi 2 daqiqasida keladi (burst 330 so'rov/s), qolgan 13 daqiqa deyarli nol. Model — 24 soatlik oyna MLP regressori (A3 5-variant kabi). Siz /forecast xizmatini yozasiz, yuklama sinovi o'tkazasiz, HPA ni burst profilida simulyatsiya qilasiz (HPA bunday profilda yomon ishlaydi), muqobil sifatida har 15 daqiqada ishga tushadigan batch Job (K8s CronJob) rejimini modellashtirasiz va ikkala rejim uchun narx va kechikishni solishtirasiz.

Model	MLP 30-128-64-1 -> ONNX (regressiya)
Xizmat turi	vaqt qatori prognozi, /forecast {window[24], features[6]}
SLO	barcha 40 000 prognoz oyna boshidan 5 daqiqa ichida tayyor
Yuklama profili	har 15 daqiqa: 2 daqiqa 330 so'rov/s, 13 daqiqa 2 so'rov/s
Xizmat vaqti (sim)	8 ms
HPA	min 1, max 10, nishon 60 %, start 20 s, scaleDown 300 s
CronJob	4 parallel pod, 2 daqiqa ishlaydi
Narx	vCPU-soat 0,045 USD

QADAMLAR

1. MLP regressorni sintetik ma'lumotda o'qitib ONNX'ga eksport qiling; FastAPI /forecast, /forecast_batch (1000 hisoblagich), /readyz, /metrics; TestClient bilan MAE tekshiruvi.
2. Yuklama sinovi: yakka parallel 1/2/4/8/16; paketli 1000 talik so'rov; 40 000 prognoz uchun ikkala rejimda vaqt.
3. K8s manifestlari: Deployment + HPA (18 qoida) va qo'shimcha CronJob manifesti (schedule */15, parallelism 4, activeDeadlineSeconds 600) — CronJob uchun 4 ta yangi qoida yozing.
4. HPA simulyatsiyasi (s5): yuklama(t) ni 15 daqiqalik davriy burst qiling (T = 1 soat), xizmat vaqti 8 ms; SLO «5 daqiqada hammasi» ni navbat orqali tekshiring; scaleDown oynasi 300 s va 60 s ni solishtiring.
5. Batch rejimi simulyatsiyasi: har 15 daqiqada 4 pod 20 s da ishga tushib 40 000 prognozni o'lchangan throughput bilan bajaradi; tugash vaqti, pod-soniya.
6. Narx taqqosi: HPA (min 1 va min 4) vs CronJob vs doimiy 6 nusxa — kunlik vCPU-soat va USD; qaysi rejim SLO ni arzonroq bajaradi.
7. Xulosa: davriy burst yuklamada reaktiv HPA nega yomon, KEDA cron-trigger nima beradi, hisoblagichlar 400 000 bo'lsa nima o'zgaradi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); HPA vs CronJob taqqos jadvali va grafigi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

6-variant. Fintech: tranzaksiya firibgarligi skoring xizmati — qattiq p99, ikki mintaqa va kanareyka relizi qiyin

Soha: Fintech (to'lovlar)

To'lov tizimi har tranzaksiyani o'tkazishdan oldin firibgarlik ballini so'raydi: p99 20 ms dan oshsa tranzaksiya tekshiruvsiz o'tkaziladi (xavf), xizmat ikki mintaqada ishlaydi va yangi model versiyasi avval 10 % trafikka (kanareyka) chiqariladi. Yuklama 500 so'rov/s, oyning oxirgi kunida 5000 so'rov/s. Model — A3 18-variantdagi MLP. Siz xizmatga model versiyasi yorlig'ini va /metrics da versiya bo'yicha hisoblagichlarni qo'shasiz, yuklama sinovida p99 ni alohida kuzatasiz, ikki Deployment (stable/canary) manifestini tekshirasiz, HPA ni oy oxiri profilida simulyatsiya qilasiz va ikki mintaqali bulut TCO sini hisoblaysiz.

Model	MLP 28-128-64-1 -> ONNX, versiyalar v1 (FP32) va v2 (INT8 statik)
Xizmat turi	anomaliya/skoring, /score {features[28]} -> {score, decision, model_version}
SLO	p99 <= 20 ms, xatolar < 0,1 %
Yuklama profili	500 so'rov/s, oy oxiri 5000 so'rov/s (30 daqiqada o'sish, 4 soat plato)
Xizmat vaqti (sim)	3 ms
Kanareyka	stable 90 % / canary 10 % (Service selector + replicas nisbati 9:1)
HPA	min 4, max 30, nishon 50 %, start 20 s
Mintaqalar	2, har birida to'liq nusxa, RTT 25 ms yaqin mintaqa

QADAMLAR

1. Ikki ONNX versiya tayyorlang (FP32 va INT8); FastAPI xizmati MODEL_VERSION env bilan ishga tushadi, /score javobida versiya, /metrics da infer_requests_total{version=...} va p99.
2. Yuklama sinovi: parallel 1/2/4/8/16/32 mijoz, p99 ni alohida ustun qilib SLO 20 ms qaysi parallellikda buziladi; v1 va v2 ni solishtiring.
3. K8s: ikkita Deployment (score-stable replicas 9, score-canary replicas 1, umumiy label app=score + version label), bitta Service, HPA faqat stable'ga; 18 qoidaga kanareyka uchun 3 ta yangi qoida qo'shing (versiya label, canary replicas <= 10 %, canary'da alohida readiness).
4. Kanareyka tahlili: TestClient bilan 1000 so'rovni 9:1 nisbatda yo'naltirib (tasodifiy marshrutlash), versiya bo'yicha p99 va qaror ulushini /metrics dan oling; v2 qarorlari v1 dan qancha farq qiladi (tranzaksiya soni).
5. HPA simulyatsiyasi (s5): oy oxiri profili, xizmat vaqti 3 ms, min 4/max 30; SLO p99 ni «p95_ms» o'rniga navbat + 2x xizmat vaqti bilan baholang; min 4 va min 10 taqqosi.
6. TCO (s6) ikki mintaqa uchun: bulut nusxalar (oddiy kun va oy oxiri), mintaqalararo trafik, 3 yil; bitta mintaqa + zaxira bilan solishtiring; mintaqa uzilsa (yiliga 4 soat) yo'qotilgan tranzaksiyalar.
7. Xulosa: kanareyka qachon 100 % ga o'tkaziladi (mezon), p99 uchun HPA yetarlimi, GPU kerakmi (yo'q — asoslang).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Kanareyka manifestlari va versiya bo'yicha metrikalar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

7-variant. Agro: issiqxona sug'orish tavsiyasi — Raspberry Pi shlyuzi simulyatsiyasi va 5 yillik TCO oson

Soha: Agro

Agrofirma 50 ta issiqxonada har 5 daqiqada sensorlar (A3 2-variant: 20 belgi, 4 sinf tavsiya) bo'yicha sug'orish qarorini oladi; internet qishloqda beqaror (kuniga 1-2 soat uziladi), qaror kechiksa ekin zarar ko'radi. Siz /advise xizmatini yozasiz va uni ikki rejimda sinaysiz: «Edge» (1 oqim + har so'rovga 12 ms sun'iy ishlov — Raspberry Pi taqlidi) va «Cloud» (2 oqim + 100 ms RTT taqlidi). Yuklama kichik (jami 0,17 so'rov/s), shuning uchun masshtablash emas, ishonchlilik va narx asosiy. Dockerfile arm64 uchun tekshiriladi, HPA simulyatsiyasi o'rniga uzilishlar simulyatsiyasi, TCO 5 yil.

Model	MLP 20-256-128-4 -> ONNX
Xizmat turi	tavsiya, /advise {sensors[20]} -> {action, confidence}
SLO	qaror 5 daqiqa ichida, oflayn ishlash
Yuklama	50 issiqxona x 1 so'rov / 5 daqiqa
Edge taqlidi	ORT 1 oqim + 12 ms sleep
Cloud taqlidi	ORT 2 oqim + 100 ms sleep (RTT)
Uzilishlar	kuniga 90 daqiqa, tasodifiy
TCO	Raspberry Pi 4 (90 USD + 5 Vt) x 50 + zaxira 5 vs bulut 1 nusxa 0,045 USD/soat + 4G modemlar

QADAMLAR

1. MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /advise, /readyz, /metrics; EDGE_MODE env bilan 12 ms yoki 100 ms sun'iy kechikish qo'shiladi.
2. Ikkala rejimda ketma-ket 200 so'rov p50/p95 va parallel 1/2/4 mijoz; sof inferens ulushi; Little qonuni tekshiruvi.
3. Dockerfile (s3): python:3.12-slim arm64 + ORT varianti 11 qoida; tasvir hajmi va 4G (5 Mbit/s) orqali 50 qurilmaga yangilash vaqti va trafik.
4. Uzilishlar simulyatsiyasi (s5 o'rniga): 7 kunlik vaqt o'qi (1 daqiqa qadam), kuniga 90 daqiqa tasodifiy uzilish; Cloud rejimida uzilish paytidagi so'rovlar yo'qoladi yoki navbatga tushadi (10 daqiqagacha); Edge rejimida yo'qolmaydi; yo'qolgan/kechikkan qarorlar sonini hisoblang.
5. TCO (s6) 1/3/5 yil: Edge (50+5 Pi, elektr, yiliga 10 % almashtirish) vs Cloud (1 nusxa + 50 modem x 3 USD/oy) vs gibrid (Pi'da qaror, bulutda monitoring); sezgirlik: issiqxona 200 ta, uzilish kuniga 4 soat.
6. Xulosa: agrofirmaga tavsiya, uzilishlar narxini (zarar ko'rgan qarorlar) TCO ga qanday qo'shish mumkin, Jetson kerakmi (yo'q — asoslang).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Uzilishlar simulyatsiyasi jadvali (yo'qolgan qarorlar)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

8-variant. Transport: avtobus bortida yo'lovchilarni sanash — Jetson taqlidi, 4G RTT va 300 avtobus TCO o'rta

Soha: Transport

Shahar avtobus parki 300 ta avtobusning eshik kamerasidan yo'lovchilar sonini hisoblaydi; kadr surrogati — digits CNN (sinf = yo'lovchilar soni 0..9). Har eshik ochilganda 2 soniya davomida 5 kadr/s, kuniga avtobus boshiga 400 to'xtash. Bulut varianti 4G orqali ishlaydi (RTT 120 ms, tunnel va chekka hududlarda uzilish), Edge varianti — bortdagi kichik kompyuter (Jetson Nano taqlidi: 1 oqim + 8 ms). Siz /count xizmatini yozasiz, ikkala rejimni sinaysiz, bulut uchun K8s manifest va HPA ni ertalabki/kechki cho'qqi profilida tekshirasiz va 300 avtobus uchun 3 yillik TCO va trafikni hisoblaysiz.

Model	kichik CNN (digits) -> ONNX, 10 sinf
Xizmat turi	tasvir, /count {pixels[64]} -> {count, confidence}
SLO	p95 <= 400 ms (bulut), oflayn ishlash istalgan
Yuklama profili	cho'qqi 07-09 va 17-19: 300 avtobus x 10 kadr / 30 s = 100 so'rov/s; kunduzi 30; tunda 2
Xizmat vaqti (sim)	5 ms
HPA	min 2, max 8, nishon 60 %
Edge	bort kompyuter 180 USD + 8 Vt, 1 oqim + 8 ms
Trafik	kadr 3 KB, 4G 1,5 USD/GB

QADAMLAR

1. CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /count, /readyz, /metrics; EDGE_MODE (1 oqim + 8 ms) va CLOUD_MODE (2 oqim + 120 ms RTT taqlidi).
2. Yuklama sinovi ikkala rejimda: parallel 1/2/4/8/16; cloud rejimida RTT tufayli little_N qanday o'sadi; SLO 400 ms tekshiruvi.
3. K8s manifest (s4) bulut xizmati uchun 18 qoida; HPA chegarasi; Dockerfile (s3) 11 qoida.
4. HPA simulyatsiyasi (s5): 24 soatlik profil (tun 2, kunduz 30, cho'qqi 100 so'rov/s, 30 daqiqalik o'tish), 1 daqiqa qadam; SLO buzilish, pod-soniya; min 2 vs min 4.
5. Trafik va TCO (s6): bulut — kuniga 300 x 400 x 10 kadr x 3 KB egress + 4G narxi + nusxalar; Edge — 300 bort kompyuter + 10 % zaxira + quvvat; gibrid — bortda sanash, bulutga faqat soatlik yig'indi; 1/3/5 yil.
6. Xulosa: avtobus parkiga tavsiya, 4G trafik ulushi TCO da qancha, uzilishlar bo'lsa bulut variantida ma'lumot yo'qolishi, Jetson haqiqiy qurilmada nimani o'lchash kerak (tegrastats).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

9-variant. Ta'lim: insholarni avtomatik baholash xizmati — imtihon kuni, sekin start va oldindan masshtablash o'rta

Soha: Ta'lim

Milliy test markazi imtihon kuni 2 soat ichida 50 000 talabaning insholarini (matn belgilari MLP, A3 15-variant kabi 1000 o'lchamli vektor) 5 ballik shkalada baholaydi; talabalar inshoni topshirgach 30 soniya ichida dastlabki ball ko'rsatilishi kerak. Yuklama imtihon boshlanishidan 90 daqiqa o'tgach keskin oshadi (ko'pchilik oxirgi 30 daqiqada topshiradi): 5 so'rov/s dan 40 so'rov/s gacha. Model tasviri katta (til resurslari bilan 1,2 GB), shuning uchun Pod start 90 s. Siz /grade xizmatini yozasiz, yuklama sinovi o'tkazasiz, HPA ni sekin start bilan simulyatsiya qilasiz va oldindan masshtablash (imtihondan 15 daqiqa oldin) hamda tasvirni kichraytirish (start 20 s) variantlarini solishtirasiz.

Model	MLP 1000-512-128-5 -> ONNX
Xizmat turi	matn baholash, /grade {vector[1000]} -> {score 1..5, confidence}
SLO	p95 <= 30 s (navbat bilan), ball yo'qolmasligi
Yuklama profili	0-90 daqiqa 5 so'rov/s, 90-120 daqiqa 40 so'rov/s
Xizmat vaqti (sim)	120 ms (matn ishlovi bilan)
HPA	min 2, max 12, nishon 60 %, start 90 s (katta tasvir) yoki 20 s (slim)
Tasvir	1,2 GB vs 280 MB
Narx	vCPU-soat 0,045 USD

QADAMLAR

1. MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /grade, /readyz (model + «til resursi» yuklash taqlidi 3 s), /metrics; TestClient bilan readyz 503 -> 200 ketma-ketligi.
2. Yuklama sinovi: parallel 1/2/4/8/16 mijoz, har so'rovga 120 ms sun'iy ishlov qo'shilgan; Little qonuni bilan N; bitta nusxa sig'imi (so'rov/s).
3. Dockerfile (s3): ikki variant — til resurslari tasvir ichida (1,2 GB) va alohida volume'dan yuklanadi (280 MB); 11 qoida va tortish vaqti 1 Gbit/s da; start vaqtiga ta'siri.
4. K8s manifest (s4) 18 qoida; readinessProbe initialDelay/failureThreshold ni 90 s startga moslang va buni qoida sifatida tekshiring.
5. HPA simulyatsiyasi (s5): imtihon profili (T = 2 soat, 1 s qadam), xizmat vaqti 120 ms; 4 konfiguratsiya: start 90 s, start 20 s, oldindan masshtab (t = 75 daqiqada 10 nusxa), min 10; SLO 30 s buzilgan vaqt, maksimal navbat, pod-soniya.
6. Narx va xulosa: har konfiguratsiya uchun imtihon kuni vCPU-soat va USD; kechikkan ballar soni; test markaziga tavsiya (oldindan masshtab + slim tasvir), GPU kerakmi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 4 konfiguratsiyali HPA taqqos jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

10-variant. Xavfsizlik: turniketda yuz embeddinglarini solishtirish — oflayn Edge va 40 turniket oson

Soha: Xavfsizlik

Korxona 40 ta turniketda xodimlarni yuz embeddingi (128 o'lchamli vektor, sintetik make_blobs bilan 500 xodim) bo'yicha tekshiradi: /verify so'rovi embeddingni oladi, bazadagi eng yaqin xodimni kosinus o'xshashlik bilan topadi va chegara bo'yicha «ruxsat/rad» qaytaradi. Ertalab 08:00-09:00 da 40 turniketdan jami 20 so'rov/s, qolgan vaqt 1 so'rov/s; javob 300 ms ichida bo'lmasa navbat hosil bo'ladi; turniket internet uzilganda ham ishlashi shart. Siz xizmatni yozasiz, yuklamani sinaysiz, Edge (turniket kontrolleri) va Cloud rejimlarini solishtirasiz, K8s manifest va HPA ni ertalabki profil bilan tekshirasiz va TCO hisoblaysiz.

Model	kosinus o'xshashlik + chegara 0,8 (sklearn make_blobs: 500 markaz, 128 o'lcham, std 0,15), ONNX o'rniga numpy matritsa
Xizmat turi	tasvir/embedding, /verify {embedding[128]} -> {employee_id, similarity, allowed}
SLO	p95 <= 300 ms, oflayn ishlash majburiy
Yuklama profili	08:00-09:00 20 so'rov/s, boshqa vaqt 1 so'rov/s
Xizmat vaqti (sim)	2 ms (Edge 1 oqim + 20 ms ARM taqlidi)
HPA	min 1, max 3, nishon 50 %
Edge	turniket kontrolleri 120 USD + 4 Vt x 40
Baza yangilanishi	kuniga 1 marta, 500 x 128 x 4 bayt

QADAMLAR

1. 500 xodim embedding bazasini make_blobs bilan yarating, test so'rovlari = markaz + shovqin (xodim) va tasodifiy vektor (begona); FastAPI /verify, /readyz, /metrics (ruxsat/rad hisoblagichlari); TestClient bilan FAR/FRR ni chegara 0,7/0,8/0,9 da o'lchang.
2. Yuklama sinovi: parallel 1/2/4/8 mijoz Edge va Cloud taqlidida (Cloud: +80 ms RTT); SLO 300 ms; Little qonuni.
3. Dockerfile (s3) arm64 slim variant 11 qoida; baza faylini volume orqali ulash (ENV + ConfigMap) — 12 omil qoidasi.
4. K8s manifest (s4) bulut varianti uchun 18 qoida; HPA min 1 bo'lsa qaysi qoida buziladi va nega bu xizmat uchun qabul qilinadi/qilinmaydi.
5. HPA simulyatsiyasi (s5): 24 soat, 1 daqiqa qadam, ertalabki cho'qqi; Cloud rejimida SLO buzilish; Edge rejimida har turniket o'z yukini (0,5 so'rov/s) ko'taradi — masshtablash kerak emasligini ko'rsating.
6. TCO (s6) 1/3/5 yil: Edge 40 kontroller + bulut sinxronizatsiya vs Cloud + 40 yupqa mijoz; internet uzilishida (yiliga 30 soat) Cloud rejimida nechta xodim kira olmaydi; xulosa va maxfiylik ustuni (embeddinglar binodan chiqadimi).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); FAR/FRR jadvali (3 chegara)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

11-variant. Logistika: ombor konveyerida paket nuqsonini aniqlash — 8 liniya, PDB va rolling update simulyatsiyasi qiyin

Soha: Logistika

Saralash ombori 8 ta konveyer liniyasida soniyasiga 20 tadan paketning tasvirini (digits surrogati, 2 sinf) tekshiradi — jami 160 so'rov/s, 24/7, qaror 50 ms ichida kerak, aks holda paket qayta aylanadi (liniya unumdorligi tushadi). Xizmat ombor ichidagi 3 tugunli k3s klasterida ishlaydi; model haftada bir yangilanadi va yangilanish paytida liniyalar to'xtamasligi shart. Siz xizmatni yozasiz, yuklamani sinaysiz, Deployment + PDB + HPA manifestlarini tekshirasiz, HPA simulyatsiyasiga rolling update (maxSurge 1 / maxUnavailable 0 vs maxUnavailable 1) va tugun nosozligini (3 tugundan biri 10 daqiqaga o'chadi) qo'shasiz, qayta aylangan paketlar sonini hisoblaysiz.

Model	kichik CNN (digits -> 2 sinf) -> ONNX
Xizmat turi	tasvir, /inspect {pixels[64]} -> {defect, confidence}
SLO	p95 <= 50 ms, qayta aylangan paketlar < 0,5 %
Yuklama profili	160 so'rov/s doimiy
Xizmat vaqti (sim)	6 ms
K8s	3 tugun, replicas 4, HPA min 4 max 9, PDB minAvailable 3, podAntiAffinity
Rollout	maxSurge 1 / maxUnavailable 0 vs maxUnavailable 1, yangi Pod 25 s
Nosozlik	t=600 s da 1 tugun 10 daqiqaga o'chadi

QADAMLAR

1. CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /inspect, /readyz, /metrics (nuqson ulushi, p95); TestClient bilan 200 paketda aniqlik.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16; bitta nusxa sig'imi; Little qonuni bilan 160 so'rov/s uchun nusxalar soni p95 <= 50 ms sharti bilan.
3. K8s manifestlari (s4): Deployment (replicas 4, RollingUpdate), PDB, HPA, Service; podAntiAffinity (tugun bo'yicha tarqatish) va PDB uchun 4 ta yangi qoida — jami 22 qoida 100 % PASS; Dockerfile 11 qoida.
4. HPA simulyatsiyasi (s5) doimiy 160 so'rov/s da: asosiy holat SLO ichida ekanini tasdiqlang (bazaviy chiziq).
5. Rolling update simulyatsiyasi: t=300 s da yangilanish, nusxalar birma-bir almashadi (yangi Pod 25 s), ikki strategiya; sig'im, navbat, p95 grafigi; qayta aylangan paketlar (p95 > 50 ms bo'lgan soniyalar x 160) soni.
6. Tugun nosozligi simulyatsiyasi: t=600 s da 1 tugun o'chadi (uning 1-2 Pod'i yo'qoladi), HPA va PDB qanday javob beradi, qayta ko'tarilish vaqti; antiAffinity bo'lmasa (barcha Pod bitta tugunda) nima bo'lishini solishtiring.
7. Xulosa: omborga tavsiya (replicas, PDB, strategiya), haftalik yangilanish qaysi vaqtda, GPU/Jetson kerakmi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Rolling update va tugun nosozligi grafiklari; 22 qoidali manifest tekshiruvi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

12-variant. Shahar: chorraha svetofor kontrollerlari uchun tiqilinch bashorati — 400 chorraha, Edge kontroller vs markaz oson

Soha: Shahar

Shahar transport markazi 400 ta chorrahadan har 2 soniyada 6x6 trafik xaritasini oladi (A3 12-variant CNN) va tiqilinch darajasini qaytaradi; natija svetofor fazasini sozlaydi, shuning uchun 500 ms ichida kelishi kerak. Yuklama doimiy 200 so'rov/s, ertalab va kechqurun 1,5 baravar. Markaziy serverga ulanish optik tola, lekin ba'zi chorrahalar radiokanal (RTT 150 ms, paketlar yo'qoladi). Siz /congestion xizmatini yozasiz, yuklamani sinaysiz, Dockerfile va manifestlarni tekshirasiz, HPA ni kunlik profilda simulyatsiya qilasiz va «hamma chorrahada kontroller» vs «markazda K8s» vs «gibrid (radiokanalli 60 chorrahada kontroller)» TCO sini hisoblaysiz.

Model	CNN 6x6 -> 3 sinf -> ONNX
Xizmat turi	vaqt qatori/tasvir, /congestion {grid[36]} -> {level, probs}
SLO	p95 <= 500 ms
Yuklama profili	200 so'rov/s, 07-09 va 17-19 da 300
Xizmat vaqti (sim)	3 ms
HPA	min 2, max 6, nishon 60 %
Edge kontroller	140 USD + 6 Vt, 1 oqim
Radiokanal	60 chorraha, RTT 150 ms, 2 % yo'qotish

QADAMLAR

1. Sintetik 6x6 xaritalarda CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /congestion, /readyz, /metrics; TestClient aniqlik.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16; 200 va 300 so'rov/s uchun nusxalar soni; radiokanal taqlidi (150 ms sleep + 2 % so'rovda 503) bilan p95 va xatolar ulushi.
3. Dockerfile (s3) 11 qoida; K8s manifest (s4) 18 qoida; HPA chegarasi millicore'da.
4. HPA simulyatsiyasi (s5): 24 soat, 1 daqiqa qadam, ikki cho'qqi; SLO buzilish, pod-soniya; nishon 60 % vs 40 %.
5. TCO (s6) 1/3/5 yil: 400 kontroller vs markaz (K8s, 3 tugun server 4000 USD har biri + elektr) vs gibrid (60 kontroller + markaz); radiokanalli chorrahalarda yo'qolgan qarorlar soni markaz variantida (2 % x so'rovlar).
6. Xulosa: shaharga tavsiya, radiokanal muammosini Edge qanday hal qiladi, 400 kontrollerni yangilash (OTA) xarajati TCO ga qanday qo'shiladi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

13-variant. Ekologiya: havo sifati prognozi — juda kam yuklama, scale-to-zero va serverless TCO o'rta

Soha: Ekologiya

Ekologiya agentligi 120 ta havo monitoring stansiyasidan daqiqada bitta o'lchov oladi va har biri uchun 6 soatlik prognoz hisoblaydi — jami 2 so'rov/s, tunda 0,5 so'rov/s; natija veb-saytda ko'rsatiladi, 3 soniya kechikish qabul qilinadi. Doimiy ishlaydigan 2 nusxa bunday yuklama uchun isrof. Siz /aqi_forecast xizmatini yozasiz, yuklamani sinaysiz, K8s manifestida minReplicas 1 va scale-to-zero (KEDA/Knative taqlidi: 5 daqiqa so'rovsiz bo'lsa 0 nusxa, sovuq start 15 s) variantlarini simulyatsiya qilasiz va doimiy pod, scale-to-zero hamda serverless funksiya (so'rov boshiga narx) TCO sini solishtirasiz.

Model	MLP 13-256-128-5 (AQI sinfi) + regressiya boshi -> ONNX
Xizmat turi	vaqt qatori prognozi, /aqi_forecast {features[13]} -> {aqi_class, values[6]}
SLO	p95 <= 3 s
Yuklama profili	kunduzi 2 so'rov/s, tunda 0,5; har 10 daqiqada 60 s so'rovsiz oraliq
Xizmat vaqti (sim)	10 ms
Scale-to-zero	5 daqiqa so'rovsiz -> 0 nusxa, sovuq start 15 s
Serverless narxi	0,0000002 USD/so'rov + 0,0000167 USD/GB-s (256 MB)
Doimiy pod	0,5 vCPU 0,0225 USD/soat

QADAMLAR

1. MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /aqi_forecast, /readyz (sovuq start taqlidi: model yuklash 2 s), /metrics; TestClient bilan birinchi so'rov (sovuq) va keyingi (iliq) kechikishini alohida o'lchang.
2. Yuklama sinovi (s2): parallel 1/2/4 mijoz (yuklama kichik), bitta nusxa sig'imi; 2 so'rov/s uchun CPU bandlik foizi (requests 500m ga nisbatan).
3. K8s manifest (s4): minReplicas 1 varianti 18 qoida (replicas >= 2 qoidasi buziladi — asoslang); Dockerfile (s3) hajmi sovuq startga qanday ta'sir qiladi.
4. Simulyatsiya (s5 o'zgartirilgan): 24 soat, 1 s qadam, 3 siyosat — doimiy 1 nusxa, doimiy 2 nusxa, scale-to-zero (5 daqiqa bo'shliqdan keyin 0, sovuq start 15 s); SLO 3 s buzilgan so'rovlar soni, pod-soniya.
5. TCO (s6) yillik: doimiy 1 va 2 pod, scale-to-zero (pod-soniya bo'yicha), serverless (so'rov soni x narx + GB-s); stansiyalar 1200 ta bo'lsa kesishish nuqtasi qayerda.
6. Xulosa: agentlikka tavsiya, sovuq start SLO ni qancha buzadi, Edge (stansiyada hisoblash) nega bu yerda ma'nosiz.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 3 siyosat taqqos jadvali (SLO, pod-soniya, USD)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

14-variant. Sport: fitnes ilovasi uchun harakat tasnifi — 1 mln foydalanuvchi, kunlik sinusoidal yuklama va on-device taqqosi qiyin

Soha: Sport

Fitnes ilovasi 1 million foydalanuvchining akselerometr oynalarini (A3 14-variant 1D CNN + GRU, 6 sinf) tasniflaydi; mashg'ulot paytida har 2 soniyada bitta so'rov. Bir vaqtda faol foydalanuvchilar kun davomida sinusoidal o'zgaradi: tunda 2000, ertalab 7:00 va kechqurun 19:00 da 60 000 (jami 30 000 so'rov/s cho'qqi). Muqobil — modelni telefonda ishlatish (on-device): server yuklamasi nolga tushadi, lekin batareya sarfi va yangilash murakkabligi paydo bo'ladi. Siz /activity xizmatini yozasiz, yuklamani sinaysiz, HPA ni sinusoidal profil bilan simulyatsiya qilasiz (nishon va oynalarni sozlab), bulut TCO ni hisoblaysiz va on-device variant bilan (telefon energiyasi A3 s6 modeli bilan) solishtirasiz.

Model	Conv1d+GRU -> ONNX (64x3 -> 6 sinf)
Xizmat turi	vaqt qatori, /activity {window[64][3]} -> {activity, probs}
SLO	p95 <= 200 ms
Yuklama profili	sinusoidal: 1000 + 29000 x (1 + sin) / 2 so'rov/s, ikki cho'qqi (12 soat davr), 1 daqiqa qadam, 24 soat
Xizmat vaqti (sim)	5 ms
HPA	min 10, max 300, nishon 50/70 %, scaleUp oynasi 0/60 s, scaleDown 300/900 s
Bulut narxi	2 vCPU nusxa 0,09 USD/soat
On-device	telefon 0,3 Vt x inferens vaqti, 1 so'rov / 2 s, 1 soat mashg'ulot

QADAMLAR

1. 1D CNN + GRU modelini sintetik akselerometrda o'qitib ONNX'ga eksport qiling; FastAPI /activity, /activity_batch, /readyz, /metrics; TestClient aniqlik.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16/32; bitta nusxa sig'imi; Little qonuni bilan cho'qqi 30 000 so'rov/s uchun nusxalar soni; paketli so'rov (telefon 10 oynani birga yuborsa) sig'imni qancha oshiradi.
3. K8s manifest (s4) 18 qoida, maxReplicas 300 uchun klaster vCPU hisobi; Dockerfile (s3).
4. HPA simulyatsiyasi (s5): sinusoidal profil, 1 daqiqa qadam (HPA davri 15 s ni mos ravishda moslang), 6 konfiguratsiya (nishon 50/70 %, scaleUp 0/60, scaleDown 300/900); SLO buzilish daqiqalari, maksimal nusxa, pod-soat; HPA sinusoidani qanchalik kechikib kuzatadi (fazaviy siljish daqiqada).
5. Bulut TCO yillik: eng yaxshi HPA konfiguratsiyasi vs doimiy 300 nusxa vs prognozli masshtab (ideal — sig'im = yuklama / 0,7); grafik.
6. On-device variant: A3 s6 energiya modeli bilan telefon uchun uJ/inferens (P_bosh 0,1 Vt, P_to'liq 0,3 Vt), 1 soatlik mashg'ulotda mAh (3,85 V, 4000 mAh batareya) va foiz; model yangilash trafigi (1 mln x hajm); server narxi 0 — lekin qaysi xarajatlar qoladi.

7. Xulosa: ilova uchun gibrid taklif (on-device tasnif + serverda yig'ma tahlil), HPA sozlamalari, GPU bu xizmatga kerakmi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 6 konfiguratsiyali HPA jadvali; Bulut vs on-device taqqos jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

15-variant. Media: yangiliklar sarlavhasini teglash xizmati — viral portlash, navbat chegarasi va 503 o'rta

Soha: Media

Yangiliklar agregatori yangi sarlavhalarni 8 ta mavzu tegiga ajratadi (matn vektori MLP, sintetik). Odatiy yuklama 50 so'rov/s, viral voqea paytida 5 daqiqa ichida 30 baravar (1500 so'rov/s) oshadi va 20 daqiqa davom etadi. Teg kechiksa yangilik lentaga «tegsiz» tushadi — bu qabul qilinadi, lekin xizmat butunlay qulab qolishi mumkin emas. Siz /tag xizmatini yozasiz, yuklamani sinaysiz, HPA simulyatsiyasiga navbat chegarasini (ortiqcha so'rovlar 503 bilan rad etiladi) qo'shasiz, scaleUp oynasi va nishonni sozlab rad etilganlar sonini kamaytirasiz va mijoz tomonida qayta urinish (retry) strategiyasining ta'sirini TestClient bilan o'lchaysiz.

Model	MLP 1000-256-8 -> ONNX
Xizmat turi	matn tasnifi, /tag {vector[1000]} -> {tags top-2, probs}
SLO	p95 <= 500 ms, rad etilganlar < 5 % (viral vaqtda)
Yuklama profili	50 -> 1500 so'rov/s (5 daqiqada), 20 daqiqa, keyin 50
Xizmat vaqti (sim)	7 ms
Navbat chegarasi	har nusxada 200 so'rov, ortig'i 503
HPA	min 2, max 25, nishon 60 %, scaleUp oynasi 0/30/60 s
Retry	mijoz 503 da 1 marta 200 ms dan keyin qayta uradi

QADAMLAR

1. MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /tag, /readyz, /metrics (rad etilganlar hisoblagichi); xizmatga navbat chegarasi qo'shing (bir vaqtdagi so'rovlar semafori, ortig'i 503).
2. Yuklama sinovi (s2): parallel 1/2/4/8/16/32 mijoz; semafor 4 bo'lganda 503 ulushi; mijozda retry (1 marta, 200 ms) bilan va retry'siz p95 va muvaffaqiyatli so'rovlar ulushi.
3. K8s manifest (s4) 18 qoida; Dockerfile (s3); HPA chegarasi.
4. HPA simulyatsiyasi (s5) navbat chegarasi bilan: har soniyada navbat > 200 x nusxa bo'lsa ortiqcha so'rovlar rad etiladi; viral profil; scaleUp oynasi 0/30/60 s va nishon 40/60 % uchun rad etilganlar soni, SLO buzilish, pod-soniya.
5. Retry ta'siri simulyatsiyada: rad etilgan so'rovlarning 100 % i 1 soniyadan keyin qaytib keladi — yuklama qanday o'sadi (retry bo'roni), rad etilganlar soni qanday o'zgaradi; eksponensial kechiktirish (1, 2, 4 s) bilan solishtiring.
6. Xulosa: agregatorga tavsiya (navbat chegarasi, HPA sozlamalari, retry siyosati), rad etish nega qulashdan yaxshi, prognozli masshtablash mumkinmi (viral oldindan ma'lum emas).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Rad etilganlar va retry bo'roni grafiklari

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

16-variant. Tibbiyot: palata EKG monitoringi — 200 bemor, hayot uchun kritik probe'lar, PDB va bedside Edge qiyin

Soha: Tibbiyot (reanimatsiya)

Kasalxona 200 ta palata monitoridan EKG oqimini (A3 16-variant 1D CNN, 3 sinf) har soniyada bir segment sifatida tahlil qiladi — 200 so'rov/s, 24/7; fibrillyatsiya aniqlanganda 2 soniya ichida signal berilishi shart, xizmat 1 daqiqadan ortiq to'xtasa hayot xavfi. Siz xizmatni yozasiz (readiness/liveness farqi, graceful shutdown — preStop paytida yangi so'rovlar qabul qilinmaydi, lekin joriylar tugaydi), yuklamani sinaysiz, Deployment + PDB + PriorityClass manifestlarini qoidalardan o'tkazasiz, HPA simulyatsiyasiga Pod nosozliklarini (tasodifiy o'lim, 2 daqiqa qayta tiklanish) va rolling update'ni qo'shasiz, bedside Edge (har monitor o'zi hisoblaydi) bilan markaziy xizmatni ishonchlilik va TCO bo'yicha solishtirasiz.

Model	1D CNN (250 nuqta -> 3 sinf) -> ONNX
Xizmat turi	vaqt qatori/anomaliya, /ecg {segment[250]} -> {class, alarm, confidence}
SLO	signal <= 2 s, mavjudlik 99,99 %, yo'qolgan segment < 0,01 %
Yuklama profili	200 so'rov/s doimiy
Xizmat vaqti (sim)	4 ms
K8s	replicas 4, PDB minAvailable 3, PriorityClass yuqori, terminationGracePeriodSeconds 30, HPA min 4 max 8
Nosozlik	har 30 daqiqada tasodifiy 1 Pod o'ladi, 60 s da qayta tiklanadi
Edge	monitor ichidagi modul 60 USD + 2 Vt x 200

QADAMLAR

1. 1D CNN'ni sintetik EKG'da o'qitib ONNX'ga eksport qiling; FastAPI /ecg, /healthz, /readyz, /metrics (alarm hisoblagichi); graceful shutdown: /admin/drain readyz'ni 503 qiladi, joriy so'rovlar tugaydi — TestClient bilan ketma-ketlikni ko'rsating.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16; 200 so'rov/s uchun nusxalar soni p95 va alarm kechikishi 2 s sharti bilan.
3. K8s manifestlari (s4): Deployment (preStop sleep 5, terminationGracePeriodSeconds 30), PDB, PriorityClass, HPA, Service; 5 ta yangi qoida (PDB, PriorityClass, preStop, grace period, antiAffinity) — jami 23 qoida 100 % PASS.
4. Dockerfile (s3) 11 qoida; tasvir hajmi va qayta tiklanish vaqtiga ta'siri.
5. Simulyatsiya (s5 kengaytirilgan): 2 soat, 1 s qadam, doimiy 200 so'rov/s; tasodifiy Pod o'limi har 30 daqiqada (60 s tiklanish); replicas 2/3/4 va PDB bor/yo'q (yangilanish paytida) uchun yo'qolgan segmentlar soni va eng uzun uzilish; rolling update t=3600 s.
6. TCO va ishonchlilik (s6): markaziy xizmat (3 tugun + zaxira) vs bedside Edge (200 modul) vs gibrid (bedside aniqlash + markazda arxiv va ikkinchi fikr); mavjudlik hisobi (ketma-ket/parallel komponentlar), 1/3/5 yil.

7. Xulosa: kasalxonaga tavsiya, nima uchun hayot uchun kritik xizmatda Edge + markaz dublikati kerak, qaysi K8s mexanizmlari mavjudlikni qanchaga oshiradi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 23 qoidali manifest tekshiruvi; Nosozlik simulyatsiyasi jadvali (yo'qolgan segmentlar)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

17-variant. Bank: chat-bot niyat tasnifi xizmati — ish vaqti profili, versiya bo'yicha metrikalar va kanareyka 10 % o'rta

Soha: Bank xizmatlari

Bank mijozlarga xizmat ko'rsatish chat-boti har xabarni 20 ta niyatdan biriga ajratadi (matn vektori MLP, sintetik); yuklama 9:00-18:00 da 120 so'rov/s, tushlikda 180, tunda 10. Javob 150 ms ichida bo'lmasa bot «bir oz kuting» deydi va mijoz qoniqishi tushadi. Yangi model versiyasi har 2 haftada chiqadi va avval 10 % trafikda sinaladi; versiyalar bo'yicha aniqlik va kechikish alohida kuzatilishi kerak. Siz xizmatga versiya yorlig'ini qo'shasiz, yuklamani sinaysiz, stable/canary manifestlarini tekshirasiz, HPA ni ish kuni profilida simulyatsiya qilasiz va «ish vaqti bo'yicha rejalashtirilgan masshtab» (CronHPA taqlidi) bilan solishtirasiz.

Model	MLP 1000-256-20 -> ONNX, v1 va v2 (v2 = boshqa seed bilan o'qitilgan)
Xizmat turi	matn tasnifi, /intent {vector[1000]} -> {intent, confidence, model_version}
SLO	p95 <= 150 ms
Yuklama profili	tun 10, 9:00-18:00 120, 13:00-14:00 180 so'rov/s; o'tishlar 15 daqiqa
Xizmat vaqti (sim)	6 ms
Kanareyka	stable 9 nusxa / canary 1 nusxa
HPA	min 2, max 12, nishon 60 %
CronHPA	8:45 da min 6, 18:15 da min 2

QADAMLAR

1. Ikki versiya MLP'ni o'qitib (seed 0 va 1) ONNX'ga eksport qiling; FastAPI MODEL_VERSION env; /intent, /readyz, /metrics da versiya yorlig'i bilan so'rovlar, xatolar, p95; TestClient bilan har versiya aniqligi.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16; SLO 150 ms; bitta nusxa sig'imi; tushlik 180 so'rov/s uchun nusxalar soni.
3. K8s manifestlari (s4): intent-stable (9) va intent-canary (1), umumiy Service, HPA stable'ga; 18 + 3 kanareyka qoidasi; Dockerfile (s3).
4. Kanareyka tahlili: TestClient bilan 2000 so'rovni 9:1 marshrutlab versiya bo'yicha aniqlik, p95 va «ishonch < 0,5» ulushini /metrics dan oling; v2 ni 100 % ga chiqarish mezonini (aniqlik >= v1 - 0,5 p.p., p95 <= v1 x 1,1) tekshiring.
5. HPA simulyatsiyasi (s5): 24 soat, 1 daqiqa qadam, ish kuni profili; CPU-HPA vs CronHPA (min 6 ish vaqtida) vs ikkalasi birga; SLO buzilish daqiqalari, pod-soat.
6. Xulosa: bankka tavsiya (CronHPA + HPA), kanareyka natijasi, hafta oxiri profili uchun nima o'zgaradi, yillik vCPU-soat narxi (0,045 USD).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Versiya bo'yicha metrikalar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

18-variant. Konchilik: konveyer tasmasi anomaliyasi — sun'iy yo'ldosh kanali, majburiy Edge va gibrid sinxronizatsiya oson

Soha: Konchilik

Uzoq hududdagi kon konveyer tasmasining 40 ta sensor tugunidan (vibratsiya + harorat, A3 8-variant 1D CNN 4 sinf) har 2 soniyada ma'lumot keladi — 20 so'rov/s. Kon internetga faqat sun'iy yo'ldosh orqali ulangan: 2 Mbit/s, RTT 600 ms, oyiga 20 GB trafik limiti, 1 GB ortig'i 8 USD. Nosozlik aniqlansa konveyer 1 soniyada to'xtatilishi kerak — bulut varianti RTT tufayli imkonsiz. Siz /belt xizmatini yozasiz, Edge (kon serveri) va bulut (RTT taqlidi) rejimlarini sinaysiz, trafikni hisoblaysiz, bitta tugunli k3s manifestini tekshirasiz va gibrid variantda bulutga nima yuborish mumkinligini (soatlik yig'indi, anomal segmentlar) trafik limiti ichida hisoblaysiz.

Model	1D CNN (256 nuqta -> 4 sinf) -> ONNX
Xizmat turi	anomaliya, /belt {signal[256], temp} -> {class, stop}
SLO	to'xtatish qarori <= 1 s, oflayn ishlash majburiy
Yuklama profili	20 so'rov/s doimiy
Xizmat vaqti (sim)	5 ms
Kanal	2 Mbit/s, RTT 600 ms, 20 GB/oy limit, 8 USD/GB ortig'i
Segment hajmi	256 x 4 bayt + sarlavha ~ 1,1 KB
Edge server	sanoat kompyuteri 1200 USD + 35 Vt, zaxira 1

QADAMLAR

1. 1D CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /belt, /readyz, /metrics (to'xtatish hisoblagichi); TestClient aniqlik va recall (nosozlik sinflari).
2. Yuklama sinovi (s2): Edge rejimi parallel 1/2/4/8; bulut taqlidi (600 ms sleep) bilan p95 — SLO 1 s qaysi parallellikda buziladi; Little qonuni bulut rejimida (little_N katta — nega).
3. Trafik hisobi: barcha segmentlar bulutga yuborilsa oyiga GB va ortiqcha to'lov; gibrid — faqat anomal segmentlar (2 %) + soatlik yig'indi (1 KB) — oyiga GB; model yangilash (hajm) trafigi.
4. Dockerfile (s3) 11 qoida; k3s manifesti (s4) bitta tugun, replicas 2, hostPath model volume, 18 qoida.
5. Simulyatsiya (s5 o'rniga): 7 kun, 1 daqiqa qadam, sun'iy yo'ldosh kanali kuniga 2 soat uziladi (ob-havo); bulut rejimida qancha so'rov javobsiz qoladi va necha marta konveyer «xavfsizlik rejimida» to'xtaydi; Edge rejimida 0.
6. TCO (s6) 1/3/5 yil: Edge (1+1 server) vs bulut (nusxa + trafik ortig'i) vs gibrid; xulosa: konga tavsiya va bulutga nima uchun faqat arxiv/qayta o'qitish ma'lumotlari ketadi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Trafik hisobi jadvali (to'liq / gibrid)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

19-variant. Oziq-ovqat: restoranlar tarmog'ida talab prognozi — 300 filial, kuniga 3 burst va 300 ta Edge tasvirini tarqatish oson

Soha: Oziq-ovqat

Tez ovqatlanish tarmog'i 300 filialda har kuni 3 marta (06:00, 11:00, 16:00) keyingi 4 soat uchun taomlar talabini prognoz qiladi (vaqt qatori MLP, sintetik: hafta kuni, soat, ob-havo, aksiya). Har burst 5 daqiqa davom etadi: 300 filial x 40 taom = 12 000 so'rov, ya'ni 40 so'rov/s; qolgan vaqt 0,2 so'rov/s. Tarmoq xizmatni markaziy bulutda yoki har filialdagi kassa kompyuterida (Edge) ishlatishi mumkin. Siz /demand xizmatini yozasiz, yuklamani sinaysiz, Dockerfile tahlilida slim va to'liq variantlarni 300 filialga tarqatish hajmi va vaqti bo'yicha solishtirasiz, K8s manifestini tekshirasiz, HPA ni 3 burst profilida simulyatsiya qilasiz va TCO hisoblaysiz.

Model	MLP 12-128-64-1 (regressiya) -> ONNX
Xizmat turi	vaqt qatori prognozi, /demand {features[12]} -> {qty}; /demand_batch (40 taom)
SLO	burst 5 daqiqada tugashi, p95 <= 1 s
Yuklama profili	3 x kuniga 40 so'rov/s (5 daqiqa), boshqa vaqt 0,2
Xizmat vaqti (sim)	5 ms
HPA	min 1, max 4, nishon 60 %, start 20 s
Tasvir	slim+ORT (~270 MB) vs pytorch (~8 GB)
Filial kanali	20 Mbit/s

QADAMLAR

1. Sintetik talab ma'lumotida MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /demand, /demand_batch, /readyz, /metrics; TestClient MAE.
2. Yuklama sinovi (s2): yakka parallel 1/2/4/8; paketli (40 taom) so'rov; 12 000 prognozni ikkala rejimda bajarish vaqti.
3. Dockerfile (s3): slim va to'liq variantlar 11 qoida; 300 filialga tarqatish: jami GB, 20 Mbit/s kanalda har filial uchun tortish vaqti, registr trafigi; haftalik model yangilanishi bilan yillik trafik.
4. K8s manifest (s4) bulut varianti 18 qoida (minReplicas 1 ni asoslang yoki 2 qiling); Dockerfile HEALTHCHECK /readyz.
5. HPA simulyatsiyasi (s5): 24 soat, 1 daqiqa qadam (HPA davri 15 s ni moslang), 3 burst; min 1 vs min 2 vs CronHPA (burstdan 3 daqiqa oldin 3 nusxa); SLO buzilish, pod-soat.
6. TCO (s6) 1/3/5 yil: bulut (HPA) vs Edge (kassa kompyuteri allaqachon bor — faqat dastur + yangilash xarajati) vs gibrid; xulosa: tarmoqqa tavsiya va 300 filialga tasvir tarqatish nega slim bo'lishi shart.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Tasvir tarqatish jadvali (300 filial)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

20-variant. Davlat xizmatlari: hujjat raqamlarini tanish — kuniga 1 mln hujjat, onlayn xizmat vs batch job TCO oson

Soha: Davlat boshqaruvi

Davlat arxivi skanerlangan hujjatlardagi raqamli maydonlarni (digits CNN, 10 sinf) tanib bazaga yozadi: kuniga 1 million hujjat, har birida 12 ta raqam — 12 mln inferens; bundan tashqari, fuqarolar portalidan kuniga 20 000 ta onlayn so'rov (javob 2 s ichida) keladi. Siz /ocr_digit va /ocr_batch (bir hujjat 12 raqam) endpointlarini yozasiz, yuklamani sinab bitta nusxa sig'imini o'lchaysiz, kunlik arxivni onlayn xizmat orqali (doimiy nusxalar) yoki tungi batch Job (K8s Job, parallelism N) orqali qayta ishlash vaqtini va narxini hisoblaysiz, manifestlarni tekshirasiz va Edge (skaner stansiyasida tanish) variantini TCO ga qo'shasiz.

Model	kichik CNN (digits) -> ONNX, 10 sinf
Xizmat turi	tasvir, /ocr_digit {pixels[64]} va /ocr_batch [12 ta]
SLO	onlayn p95 <= 2 s; arxiv 1 mln hujjat 8 soatlik tun oynasida
Yuklama profili	onlayn 0,25 so'rov/s (kunduzi 0,6), arxiv 12 mln inferens/kun
Xizmat vaqti (sim)	2 ms / raqam
K8s Job	parallelism 2/4/8, har pod 2 vCPU
Narx	vCPU-soat 0,045 USD
Skaner stansiyalari	20 ta, har birida kompyuter bor

QADAMLAR

1. CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /ocr_digit, /ocr_batch (12 raqam, paket inferens), /readyz, /metrics; TestClient bilan 200 hujjatda raqam bo'yicha va hujjat bo'yicha (12 tasi ham to'g'ri) aniqlik.
2. Yuklama sinovi (s2): yakka va paketli so'rovlar parallel 1/2/4/8/16; bitta nusxa sig'imi (hujjat/s); 1 mln hujjat uchun 1 nusxada soatlar.
3. K8s manifestlari (s4): onlayn Deployment + HPA (18 qoida) va alohida Job manifesti (parallelism, completions, backoffLimit, activeDeadlineSeconds) — Job uchun 4 ta qoida; Dockerfile (s3).
4. Batch hisobi: parallelism 2/4/8 uchun tungi oynada tugash vaqti (o'lchangan throughput asosida) va vCPU-soat; 8 soat oynasiga sig'adigan minimal parallelism.
5. HPA simulyatsiyasi (s5): arxiv oqimi onlayn xizmatga kunduzi 10 soat davomida tekis yuborilsa (12 mln / 36 000 s = 333 inferens/s) va onlayn so'rovlar bilan aralashsa — onlayn p95 SLO buziladimi; ajratilgan Job bilan solishtiring.
6. TCO (s6) yillik: doimiy nusxalar (kunduzi arxiv + onlayn) vs onlayn kichik + tungi Job vs Edge (20 skaner stansiyasida tanish, bulutga faqat natija); xulosa: arxivga tavsiya va onlayn/batch ajratish nega to'g'ri.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Batch Job hisobi

jadvali (parallelism 2/4/8)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20